

Table of Contents

NewModula-2 - User Guide & Programming Reference	2
Getting Started	6
Lexical Structure	7
Modules & Compilation	11
Declarations & Types	16
Expressions & Operators	22
Statements & Control Flow	30
Procedures	35
Objects & Classes	40
The Standard Environment	46
Memory & Exceptions	55
Reference	60
Grammar & Syntax Summary	62

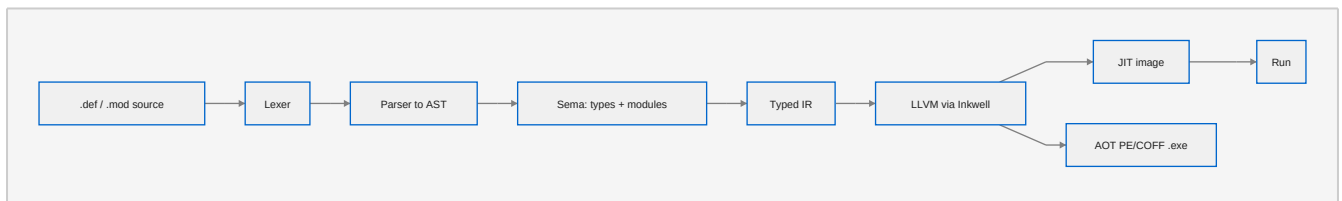
NewModula-2 - User Guide & Programming Reference

An offline manual for the NewM2 compiler: Niklaus Wirth's Modula-2 (PIM 4 + ISO 10514-1), reimaged on a modern Rust + LLVM, JIT-first architecture, with a COM-native object system for Windows.

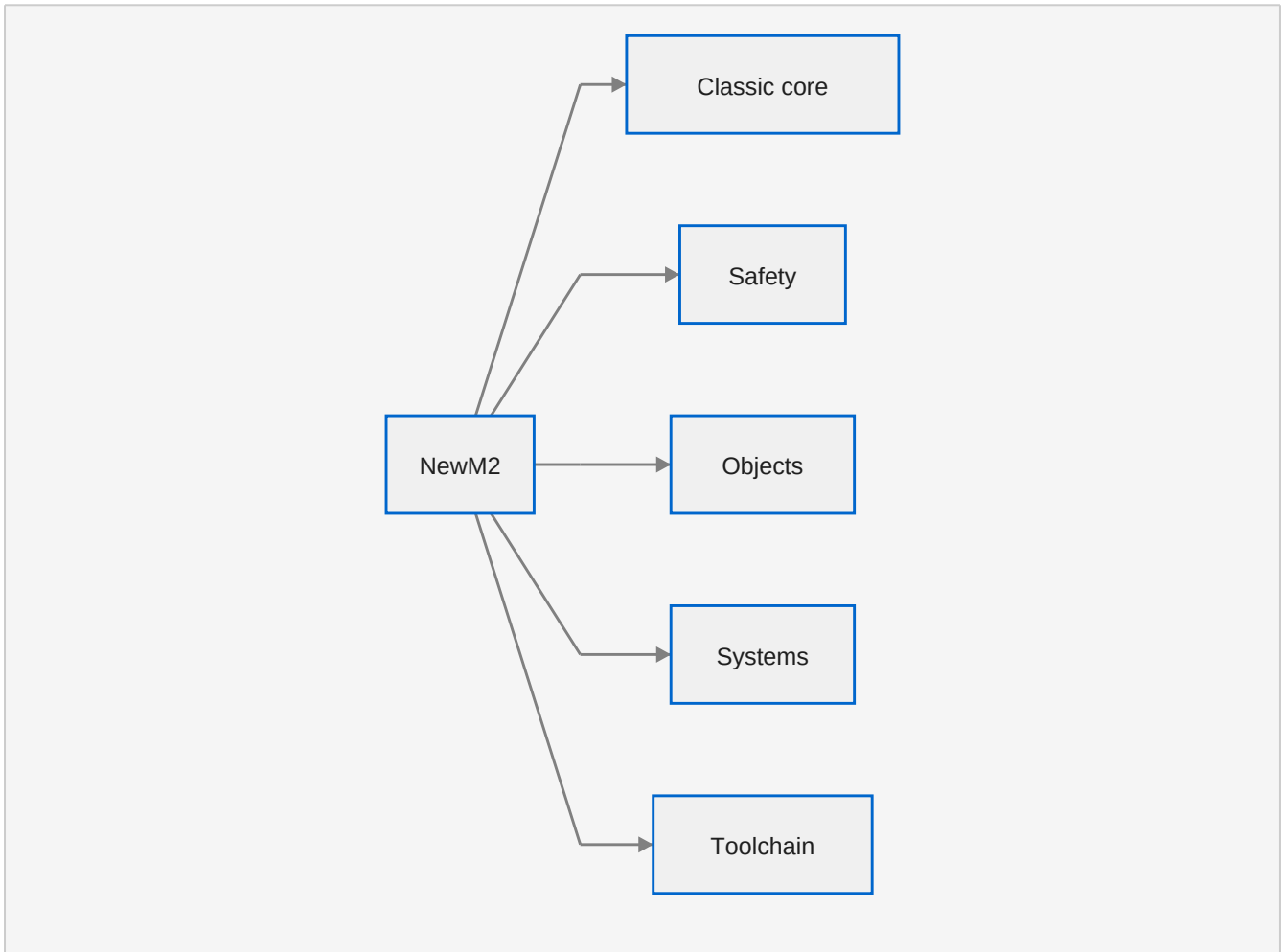
About this manual

This is a single, self-contained reference for the language **as NewM2 implements it** - both a **user guide** (Part I, a guided tour from "hello world" to the object system) and a **programming reference** (Part II, the systematic tables and grammar). Every construct is grounded in the compiler source, and each page notes where something is *live* versus still being wired.

NewM2 compiles a `.def/.mod` pair to a **memory-resident image you can run directly** (`newm2 run Hello.mod`) or to a **native PE/COFF .exe** (`newm2 build Hello.mod -o Hello.exe`).



NewM2 at a glance



Highlights of the current language:

- * **Modules with separate interface and body** - **DEFINITION** + **IMPLEMENTATION**, compiled apart.
 - * **A COM-native object layer** - single-inheritance classes whose layout is the COM vtable ABI, **INTERFACE** types with IIDs, and **GUARD / ISMEMBER** for safe runtime type discrimination. NewM2 deliberately **departs from ISO 10514-3** here to fully support COM.
 - * **Deterministic memory** - manual **NEW/DISPOSE** plus a runtime heap guard and a static leak/**DISPOSE** analysis.
 - * **Real systems programming** - the **SYSTEM** module, inline assembler, and direct calls into Win32 / COM from M2.
-

Part I - User Guide

#	Chapter	What it covers
1	Getting started	Build, run, the dump-* driver, the .def/.mod model
2	Lexical structure	Comments, identifiers, reserved words, literals, pragmas
3	Modules & compilation	DEFINITION/IMPLEMENTATION, IMPORT/EXPORT, opaque types
4	Declarations & types	CONST/TYPER/VAR, the type system
5	Expressions & operators	Arithmetic, DIV/MOD/REM, sets, precedence
6	Statements & control flow	IF, CASE, WHILE, REPEAT, FOR, LOOP, WITH
7	Procedures	Value/VAR parameters, function procedures, nesting, procedure types
8	Objects & classes	Classes, inheritance, abstract classes, interfaces/COM, GUARD/ISMEMBER
9	The standard environment	Pervasive procedures, the SYSTEM module, key libraries
10	Memory & exceptions	Pointers, NEW/DISPOSE, manual memory, EXCEPT/FINALLY

Part II - Programming Reference

#	Chapter	What it covers
11	Reference	Reserved words, pervasive identifiers, the SYSTEM module, operators
12	Grammar & syntax summary	The grammar at a glance - declarations, types, statements, classes

Conventions

- * Code is Modula-2 (`.mod/.def`) unless noted; examples are small and runnable.
- * A `src/newm2-.../...rs:NN` reference points into the NewM2 compiler source.
- * Comments are (`* ... *`) and nest. Statements are **separated** by `;`. Assignment is `:=`; equality is `=`. Reserved words and standard identifiers are UPPER CASE.
- * Diagrams are Mermaid, rendered natively by DocCrate; this manual was produced offline with `doc-crate.exe --export-pdf`.

Getting Started

How to build NewM2, run a Modula-2 program, and watch it move through the compiler.

Prerequisites

- * **Windows x86-64 / MSVC** - NewM2 targets `x86_64-pc-windows-msvc`.
- * **Rust** (stable) - from rustup.rs.
- * **LLVM 22.x** - configured in the workspace `.cargo/config.toml`.

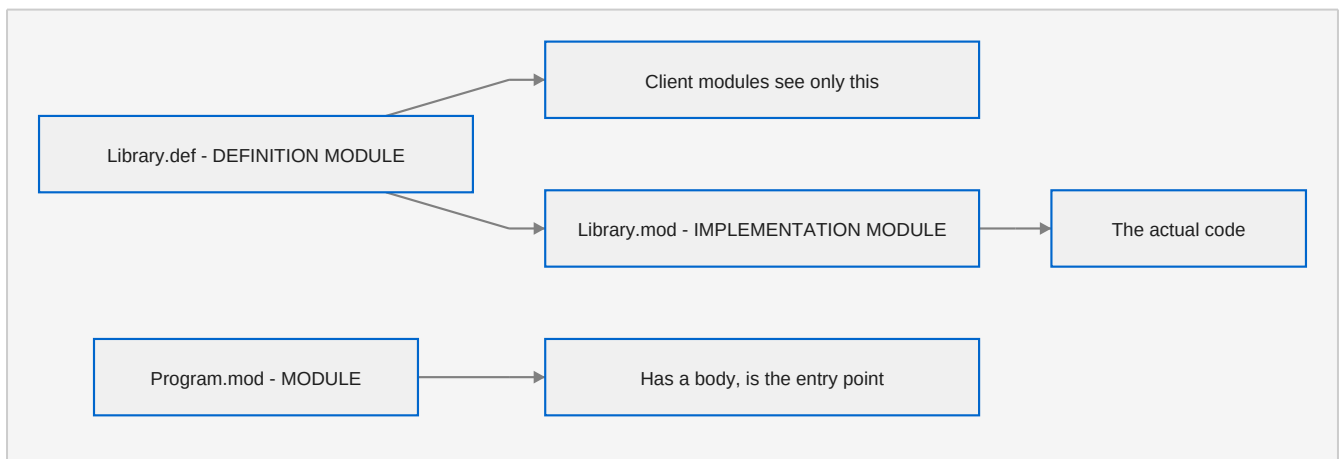
Build and run

```
cargo build --workspace
cargo run -p newm2-driver -- run Hello.mod
```

`run` JIT-compiles the module into a memory-resident image and executes it. Once the driver is on `PATH` this shortens to `newm2 run Hello.mod`. For a standalone executable, `newm2 build Hello.mod -o Hello.exe` emits a native PE/COFF `.exe`.

The .def / .mod model

Modula-2 splits a library into two files that are compiled separately:



- * A **program** is a **MODULE** (a `.mod` file) with a body - it is what you run.
- * A **library** is a **DEFINITION MODULE** (`.def` - the public interface) plus an **IMPLEMENTATION MODULE** (`.mod` - the body). Clients import only what the `.def` exports. See [Modules & compilation](#).

Watching the pipeline

The driver stops after any phase, so you can inspect exactly what the compiler produced:

Command	Shows
<code>newm2 dump-tokens f.mod</code>	the lexer's token stream
<code>newm2 dump-ast f.mod</code>	the parsed syntax tree
<code>newm2 dump-sema f.mod</code>	name/type resolution and the symbol table
<code>newm2 dump-ir f.mod</code>	the typed intermediate representation
<code>newm2 dump-llvm f.mod</code>	the generated LLVM IR
<code>newm2 dump-asm f.mod</code>	the emitted machine code
<code>newm2 run f.mod</code>	JIT-compile and execute
<code>newm2 build f.mod -o f.exe</code>	AOT-compile to a native .exe
<code>newm2 check f.mod</code>	parse + sema only

Memory: manual allocation

NewM2 uses **classical manual memory management**: `NEW` allocates via `HeapAlloc` and `DISPOSE` frees via `HeapFree`. Every allocation must be paired with a `DISPOSE` before the pointer goes out of scope. See [Memory & exceptions](#).

NewM2's front-end (lexer, parser) accepts a broad swath of PIM 4 + ISO 10514-1; the back-end is younger, so some constructs parse before they run. Each page notes status where it matters.

[NewM2 Guide home](#) ? [Lexical structure](#) ? [Modules & compilation](#)

Lexical Structure

How NewM2's lexer turns source text into tokens: comments, case-sensitive identifiers, the reserved-word set, and the literal forms. A defining Modula-2 trait shows up here - the built-in type and procedure names are *identifiers*, not keywords.

Comments and case

Comments are delimited by (*** and ***) and **nest**, so you can comment out a region that already contains a comment:

```
(* outer (* inner *) still open here *)
```

Modula-2 is **case-sensitive**, and reserved words and standard identifiers are spelled in **UPPER CASE**. **Count**, **count**, and **COUNT** are three different identifiers; **BEGIN** is a keyword but **Begin** is an ordinary identifier.

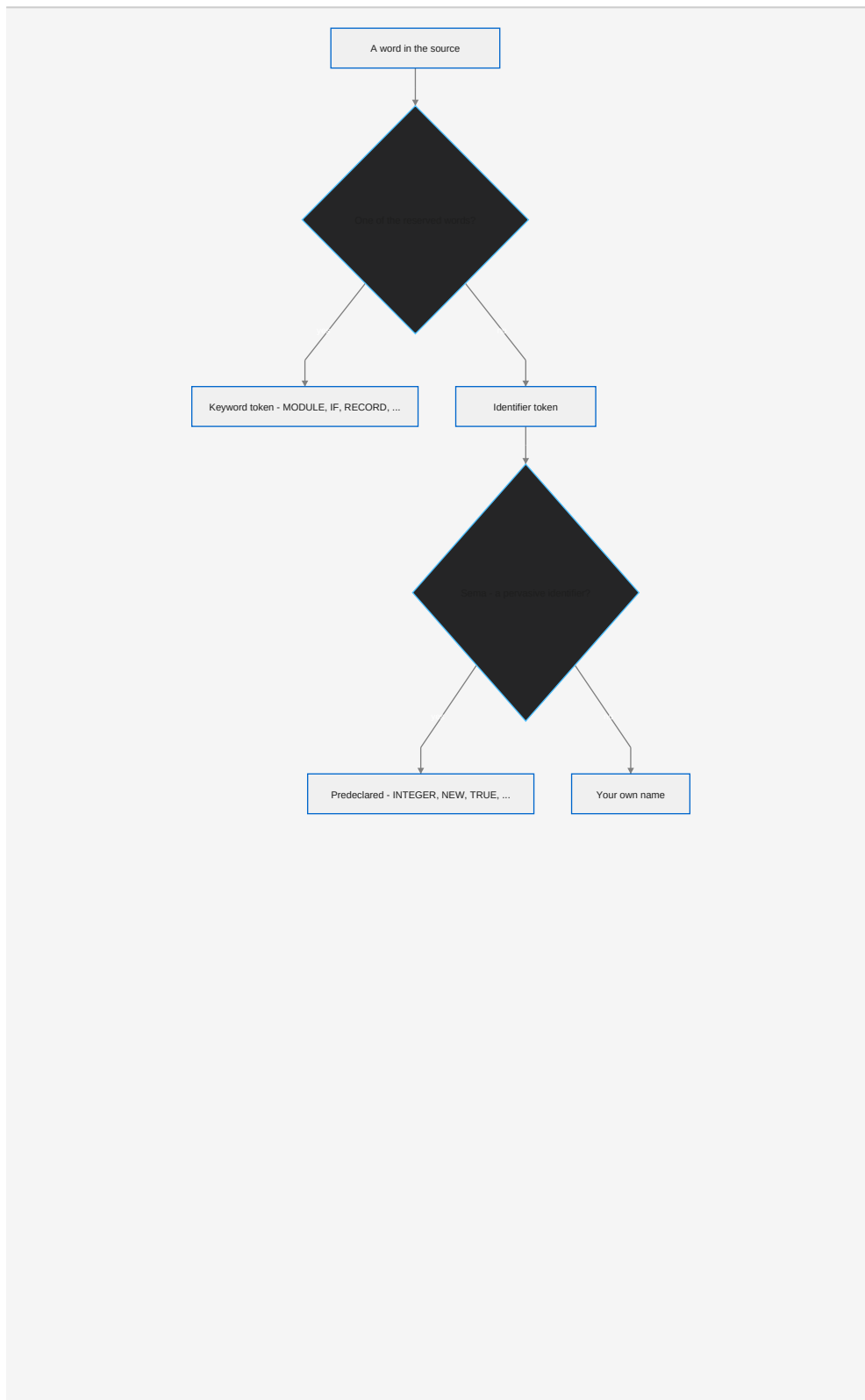
Identifiers and reserved words

An identifier is a letter followed by letters and digits. The lexer reserves the words below ([src/newm2-lexer/src/token.rs](#)) - the PIM 4 core, the ISO 10514-1 additions, a piece of the ISO 10514-2 object extension, and NewM2 extensions:

Group	Reserved words
Modules & structure	MODULE DEFINITION IMPLEMENTATION IMPORT FROM EXPORT QUALIFIED BEGIN END
Declarations	CONST TYPE VAR PROCEDURE FORWARD RECORD ARRAY SET POINTER OF TO
Control flow	IF THEN ELSIF ELSE CASE WHILE DO REPEAT UNTIL FOR BY LOOP EXIT WITH RETURN
Operators (word)	AND OR NOT DIV MOD REM IN
ISO 10514-1	EXCEPT FINALLY RETRY GENERIC PACKEDSET
ISO 10514-2 (parsed, deferred)	ABSTRACT
NewM2 extensions	ASM BAND BOR BXOR BNOT SHL SHR

Pervasive identifiers - not keywords

This is the part newcomers miss. The built-in **types** (`INTEGER`, `CARDINAL`, `REAL`, `CHAR`, `BOOLEAN`, `BITSET`, ...), the standard **procedures** (`NEW`, `DISPOSE`, `INC`, `DEC`, `INCL`, `EXCL`, `HIGH`, `SIZE`, `ORD`, `CHR`, `VAL`, `MIN`, `MAX`, `ABS`, `CAP`, ...), and the constants (`TRUE`, `FALSE`, `NIL`) are **pervasive identifiers** - predeclared in an enclosing pseudo-scope, *not* reserved words. The lexer returns them as ordinary **Ident** tokens; sema resolves them.



Because they are not reserved, you *could* declare a local `INTEGER` - Modula-2 lets you, and it shadows the pervasive one. (Don't.)

Literals

```
CONST
  dec  = 1995;      (* decimal integer *)
  oct  = 777B;      (* octal integer, B suffix *)
  hex  = 0FFH;      (* hex integer, H suffix, leading digit *)
  pi   = 3.14159;   (* REAL - needs a digit each side of the point *)
  big  = 6.022E23;  (* REAL with exponent *)
  ch   = "A";       (* character or one-char string *)
  nl   = 12C;       (* character by octal ordinal, C suffix *)
  msg  = "hello";   (* string literal *)
```

Strings may use double or single quotes ("`hi`" or '`hi`'); the other quote may appear inside unescaped. NewM2 carries `A/U` literal flavours on character and string literals (narrow/wide), and **pragmas** are written `<* ... *>` (`src/newm2-lexer/src/token.rs`).

[NewM2 Guide home](#) ? [Declarations & types](#) ? [Expressions & operators](#)

Modules & Compilation

Modula-2's defining characteristic: a library is split into a *public interface* and a *private body* that are compiled **separately**, so clients can never see - or depend on - implementation details.

Three kinds of module

Modula-2 has three distinct module kinds, reflected in the `ModuleKind` enum (`src/newm2-parser/src/ast.rs:29`):

Kind	Keyword	File	Purpose
Program	MODULE	.mod	Has a body; the entry point you run
Definition	DEFINITION MODULE	.def	Public interface - what clients see
Implementation	IMPLEMENTATION MODULE	.mod	The body - paired with a .def

There is a fourth variant, `Local`, for modules nested inside procedures; see the [Local modules](#) section below.

DEFINITION and IMPLEMENTATION

A **DEFINITION MODULE** declares everything that the outside world may use: constants, types, variables, and procedure *headers* (signature only, no body). A matching **IMPLEMENTATION MODULE** supplies the procedure bodies, private types, and any module-level initialisation code.

Here is the real pair from `Mod/tests/T30030ScalarHelper.def` and `Mod/tests/T30030ScalarHelper.mod`:

T30030ScalarHelper.def - the interface

```
DEFINITION MODULE T30030ScalarHelper;

CONST
  BaseInt = 11;
  BaseCard = 7;

PROCEDURE BumpInt(n : INTEGER) : INTEGER;
PROCEDURE BumpCard(n : CARDINAL) : CARDINAL;
PROCEDURE NextChar(ch : CHAR) : CHAR;
PROCEDURE Flip(flag : BOOLEAN) : BOOLEAN;

END T30030ScalarHelper.
```

T30030ScalarHelper.mod - the body

```
MODULE T30030ScalarHelper;

PROCEDURE BumpInt(n : INTEGER) : INTEGER;
BEGIN
    RETURN n + 4;
END BumpInt;

PROCEDURE BumpCard(n : CARDINAL) : CARDINAL;
BEGIN
    RETURN n + 9;
END BumpCard;

(* ... remaining procedure bodies ... *)

BEGIN
END T30030ScalarHelper.
```

Key points:

- * The `.def` says **DEFINITION MODULE**; the `.mod` says just **MODULE** (program or implementation - the keyword **IMPLEMENTATION** is optional in the body file by PIM 4 but is accepted by NewM2 when present).
- * Every name declared in the `.def` is **automatically exported** - you need no **EXPORT** list. A client importing `T30030ScalarHelper` sees `BaseInt`, `BaseCard`, `BumpInt`, and all the declared procedures.
- * The `.mod` body may contain **private** declarations (types, variables, helper procedures) that are invisible to importers - they simply do not appear in the `.def`.

Importing

IMPORT M; - qualified access

IMPORT names a module; its exported names are then accessed as **M.X**.

```
MODULE Example;
IMPORT SWholeIO, STextIO;

BEGIN
    SWholeIO.WriteInt(42, 0);
    STextIO.WriteLine;
END Example.
```

The dot notation prevents name clashes across modules. This is the style used throughout NewM2's own test corpus (e.g. `Mod/tests/t-30-050-deep-import-graph.mod`).

FROM M IMPORT X, Y; - unqualified access

`FROM ... IMPORT` brings specific names directly into scope so they can be used without the module prefix:

```
MODULE Example;
FROM T30020Helper IMPORT Base, WriteValue;

BEGIN
  WriteValue(Base);          (* no module prefix needed *)
  WriteValue(Base + 5);
END Example.
```

This is the live corpus example from `Mod/tests/t-30-020-import-module.mod`. The `Import` variant in the AST is `Import::From { module, names, .. }` (`src/newm2-parser/src/ast.rs:40`).

IMPORT alias := Mod; - ISO alias form

The plain `IMPORT` variant also carries an optional alias (`ImportName.alias`, `src/newm2-parser/src/ast.rs:48`), letting you rename a module at import time:

```
IMPORT SC := T30030ScalarHelper;
(* use as SC.BumpInt, SC.BaseInt ... *)
```

NewM2 parses this form; whether the alias is fully resolved through sema depends on the phase at which you are reading this.

EXPORT QUALIFIED - PIM-2 legacy

In PIM 2, a library module needed an explicit `EXPORT QUALIFIED` list. PIM 4 dropped this (definition modules export everything they declare automatically). NewM2 parses the form (`Decl::Export { qualified: bool, .. }`, `src/newm2-parser/src/ast.rs:64`) for compatibility with older source, but the corpus contains no uses of it.

Opaque types - information hiding

An **opaque type** is a type name that appears in a `.def` with *no* representation:

```
TYPE
  SharedMemory;
```

Clients see that `SharedMemory` exists and can declare variables of that type and pass them to the module's procedures, but they cannot inspect fields, dereference pointers, or construct values - the representation is entirely hidden.

The actual representation lives only in the matching `.mod`:

```
(* Inside MemShare.mod *)
SharedMemory = POINTER TO ShareData;
ShareData =
    RECORD
        map    : HANDLE;
        ...
    END;
```

This is a real pair from `library/advapidef/MemShare.def` (line 23) and `library/advapimod/MemShare.mod`. `PipedExec` uses the same pattern with `PipedExecHandle` (`library/advapidef/PipedExec.def`, line 19).

In the AST, `TypeDecl.def` is `None` for an opaque declaration (`src/newm2-parser/src/ast.rs:126` - the comment reads "None indicates an opaque type"). The definition module publishes the name; the implementation module supplies the `= ... body`; and sema reconciles the two.

Why it matters. Callers that import `MemShare` hold a `SharedMemory` value and call `AllocateSharedMemory`, `LockSharedMemory`, `DeallocateSharedMemory` - they never touch the internal `RECORD` fields. The implementation is free to change the layout without recompiling any client, as long as the procedure signatures stay the same.

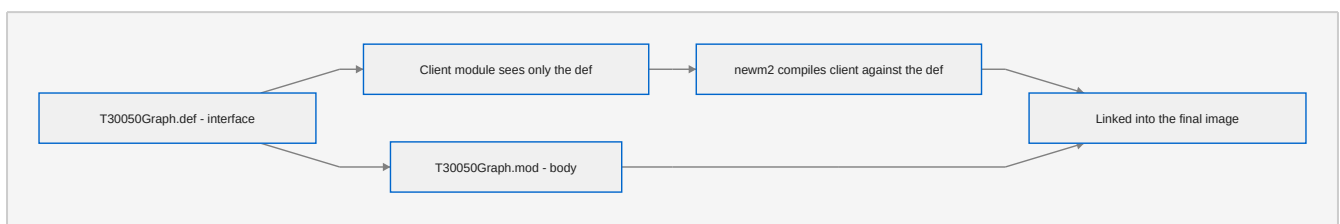
Local modules

A `MODULE ... END` block nested inside a procedure body or implementation module is a **local module**. It creates its own scope and can have its own `IMPORT` and (in PIM 2) `EXPORT QUALIFIED` clauses.

NewM2 fully parses local modules. The AST variant is `ModuleKind::Local` (`src/newm2-parser/src/ast.rs:35`) and they appear as `Decl::LocalModule(Box<Module>)` in the declaration list (`src/newm2-parser/src/ast.rs:62`). Sema opens a `ScopeKind::LocalModule` for them (`src/newm2-sema/src/scope.rs:36`). The loader also enumerates local module names from procedure bodies (`src/newm2-loader/src/graph.rs`, cited in `docs/module-graph.md`).

Local modules are not common in modern Modula-2 code and no example exists in the current corpus; they are included here for completeness.

Separate compilation and the module graph



NewM2 discovers the full set of modules to compile by building a **module graph**

before sema runs.

The algorithm (documented in [docs/module-graph.md](#)) is:

1. Parse the entry file's AST.
2. Read its **IMPORT** clauses.
3. For each imported name, find the **.def** on the search path.
4. Parse that **.def**, then recurse into *its* imports.
5. Repeat until the worklist is empty.
6. Topologically sort with cycle detection; sema then processes modules in dependency order.

The graph is **static and closed**: Modula-2 has no runtime module-loading mechanism, so every module the program can ever use must be reachable as an **IMPORT** target at compile time. This is true for both execution modes - **newm2 run** (JIT) loads all transitively required modules at session start; **newm2 build** resolves the whole graph at link time.

The live test [Mod/tests/t-30-050-deep-import-graph.mod](#) exercises a three-hop chain:

```
T30050DeepImportGraph -> T30050Graph -> T30050Mid -> T30050Leaf
```

newm2 dump-module-graph t-30-050-deep-import-graph.mod prints this graph. The [hello_module_graph_snapshot](#) compatibility test pins a known-good 5-module result through [IOChan](#).

Implementation entry points:

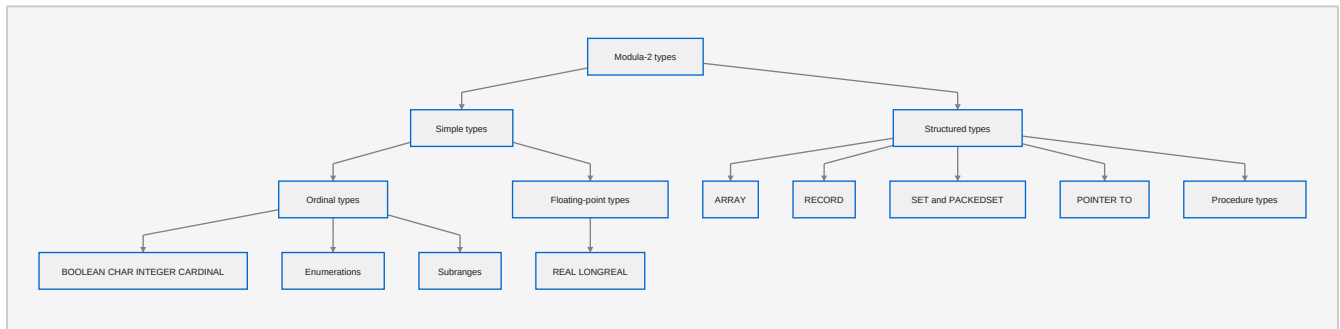
Source file	Role
src/newm2-loader/src/loader.rs	build_module_graph - BFS over IMPORT clauses with Tarjan cycle detection
src/newm2-loader/src/graph.rs	ModuleNode - carries parsed AST, DEF/IMPL paths, content hash
src/newm2-loader/src/search_path.rs	DEF lookup along search path; IMPL sibling-dir heuristic
src/newm2-loader/src/cache.rs	Phase 10 incremental cache (scaffolded in Phase 2)

[NewM2 Guide home](#) ? [Getting started](#) ? [Declarations & types](#) ? [Procedures](#)

Declarations & Types

How NewM2 names things: the three declaration sections, the built-in scalar types, and the full range of structured types that Modula-2 lets you compose from them.

The type taxonomy



Ordinal types have a total ordering and can be used as array indices, **CASE** selectors, **SET** element types, and subrange hosts. Floating-point types are not ordinal.

CONST, TYPE, VAR

A declaration block in a module body or procedure body may contain any combination of **CONST**, **TYPE**, and **VAR** sections, in any order, repeated as many times as needed. Each section ends when the next keyword (**CONST**, **TYPE**, **VAR**, **PROCEDURE**, **BEGIN**, **END**) is encountered. This is intentional: you can open a new **CONST** after a **TYPE** section if the constants depend on a type you just defined.

```
CONST
  MaxItems = 256;
  Version  = "1.0";

TYPE
  Index    = [0..MaxItems-1];
  Table    = ARRAY Index OF INTEGER;

VAR
  count : INTEGER;
  data  : Table;
```

Constant expressions. The expression on the right-hand side of a **CONST** declaration must be a compile-time constant: integer, real, character, or string literals, **TRUE**, **FALSE**, **NIL**, a previously declared constant, or any arithmetic or relational combination of those. **MAX**, **MIN**, **SIZE**, **ORD**, and **CHR** are also usable in constant expressions. The corpus uses expressions like `Scale / 2` and `10000` freely (see `library/advapidef/Money.def`).

VarDecl address binding. NewM2 also parses `VAR x [addr] : T;` - binding a variable to a fixed memory address, a NewM2 extension ([src/newm2-parser/src/ast.rs](https://github.com/llvm/llvm-project/blob/main/newm2-parser/src/ast.rs), `VarDecl`). This is a low-level hardware-mapping facility; it parses but back-end support is limited.

Built-in types (pervasive identifiers)

The type names below are *pervasive identifiers*, not reserved words - the lexer returns them as ordinary `Ident` tokens and sema resolves them. This means you could (but shouldn't) declare a local `INTEGER`. See [Lexical structure](#) for how the lexer/sema split works.

PIM base types

Type	Description
INTEGER	Signed machine-word integer (platform width)
CARDINAL	Unsigned machine-word integer; distinct from INTEGER
REAL	Single-precision floating-point
LONGREAL	Double-precision floating-point
BOOLEAN	TRUE or FALSE; ordinal
CHAR	A single character; ordinal
BITSET	SET OF [0..WordBits-1]; the predeclared set type
PROC	PROCEDURE with no parameters and no result (a callable value)
LONGINT	Long signed integer (wider than INTEGER on 32-bit targets)
LONGCARD	Long unsigned integer

Exact-width types

NewM2 registers these in sema ([src/newm2-sema/src/analyze.rs](#), lines 401-408) and the library uses them extensively:

Type	Width / sign
INTEGER8 INTEGER16 INTEGER32 INTEGER64	Exact-width signed
CARDINAL8 CARDINAL16 CARDINAL32 CARDINAL64	Exact-width unsigned
BYTE WORD DWORD QWORD	Storage-unit aliases
ACHAR	ANSI/narrow character (8-bit)
UCHAR	Unicode/wide character (16-bit)
ADDRESS	Pointer-width unsigned (equivalent to SYSTEM.ADDRESS)

These are pervasive identifiers; no import is needed. The `Money` module uses `INTEGER64` for its currency type, and `Float.def` uses all three widths of `INTEGER` for rounding conversion procedures.

ISO 10514-1 numeric types

`COMPLEX` and `LONGCOMPLEX` are registered in the pervasive scope ([src/newm2-sema/src/types.rs](#), `Builtin::Complex` / `Builtin::LongComplex`). They parse and type-check; full arithmetic lowering is a work in progress.

Enumerations and subranges

Enumerations introduce a new ordinal type by listing its values in parentheses. Ordinal values are 0, 1, 2, ... in declaration order:

```
TYPE
  Colour = (Red, Green, Blue);
  Direction = (North, East, South, West);
```

`ORD(Green)` is 1. `VAL(Colour, 2)` is `Blue`. Enumerations are their own nominal types: `Colour` and `Direction` are not compatible even though both have four (or three) elements. AST node: `TypeExpr::Enum` ([ast.rs](#), line 203).

Subranges constrain an ordinal type to a contiguous range. The host type is inferred from the bounds - integer literals give an `INTEGER` subrange, a character literal gives a `CHAR` subrange, an enumeration constant gives an enumeration subrange:

```

TYPE
  Digit    = [0..9];
  Letter   = ['A'..'Z'];
  PriColor = [Red..Blue];    (* subrange of Colour *)

```

Subranges are ordinal and can themselves be used as array indices or **SET** element types. AST node: `TypeExpr::Subrange` (`ast.rs`, line 202). The corpus uses them for array index types in the Win32 headers: `ARRAY [0..CCHCCTEXT-1] OF ACHAR` (`CUSTCNTL.def`).

Arrays

Fixed arrays declare an index type (typically a subrange or named ordinal) and an element type. The index may be a multi-dimensional list:

```

TYPE
  Row      = ARRAY [0..7] OF INTEGER;
  Matrix   = ARRAY [0..3], [0..3] OF REAL;
  Grid     = ARRAY Digit, Digit OF CHAR;

```

The AST holds a `Vec<TypeExpr>` for the index types and a single `Box<TypeExpr>` for the base: `TypeExpr::Array(indices, base, span)` (`ast.rs`, line 207). A multi-dim array is equivalent to an array of arrays; elements are accessed with comma-separated indices `m[i, j]` or chained `m[i][j]`.

Open arrays appear only in formal parameter position. The syntax is `ARRAY OF T` with no index bounds. The high bound is retrieved at runtime with `HIGH`:

```

PROCEDURE Sum(a : ARRAY OF INTEGER) : INTEGER;
VAR i, s : INTEGER;
BEGIN
  s := 0;
  FOR i := 0 TO HIGH(a) DO
    s := s + a[i];
  END;
  RETURN s;
END Sum;

```

AST node: `TypeExpr::OpenArray(base, span)` (`ast.rs`, line 209). Sema enforces that open arrays appear only in parameter position.

Records and variants

A **RECORD** groups named fields of different types under one type name:

```

TYPE
  Pair = RECORD
    left  : INTEGER;
    right : CARDINAL;
    mark  : CHAR;
  END;

```

This exact pattern appears in `Mod/tests/T40020RecordHelper.mod`. Records are value types - `VAR p : Pair` allocates `Pair` inline on the stack; `POINTER TO Pair` puts it on the heap.

Variant records extend a record with a `CASE` part, selecting which fields are present based on a tag value:

```

TYPE
  Shape =
    RECORD
      x, y : INTEGER;      (* fixed fields *)
      CASE kind : ShapeKind OF
        Circle : radius : INTEGER; |
        Rect   : width, height : INTEGER; |
        Point  : (* no extra fields *)
      END;
    END;

```

The tag field (`kind`) is part of the record and holds the current variant. The `ELSE` arm (`else_arm: Option<Vec<RecordField>>`) catches any tag value not listed explicitly. Nested `CASE` within a variant arm is also parsed. A real corpus example appears in `library/advapidef/DlgShell.def` (line 630):

```
CASE ct : ControlType OF PushButton: ... | RadioGroup: ...
```

AST nodes: `RecordType`, `VariantPart`, `VariantArm` (`ast.rs`, lines 221-253).

Note. The variant record parses and sema resolves field references inside each arm. Generating correct LLVM IR for the overlapping-field layout is complete for simple variants; deeply nested variants are parsed but may not yet lower fully.

Sets

`SET OF T` creates a set type over an ordinal base type. `BITSET` is the predeclared set of `[0..WordBits-1]`. `PACKEDSET OF T` is the ISO 10514-1 compact variant:

```

TYPE
  ExecFlags  = (ExecDetached, ExecMinimized, ExecMaximized, ExecHidden);
  ExecFlagSet = SET OF ExecFlags;

```

This appears verbatim in `library/advapidef/PipedExec.def`. Set variables support the operators `+` (union), `-` (difference), `*` (intersection), `/` (symmetric difference), `IN` (membership), `INCL`, and `EXCL`.

```

VAR flags : ExecFlagSet;
BEGIN
  flags := ExecFlagSet{ExecMinimized, ExecHidden};
  INCL(flags, ExecDetached);
  IF ExecMinimized IN flags THEN ... END;
END;

```

AST node: `TypeExpr::Set { packed, element, span }` (`ast.rs`, line 217). `Float.def` demonstrates `PACKEDSET: FPEceptions = PACKEDSET OF FPEception`.

Pointers

`POINTER TO T` is a heap pointer. The value `NIL` is the null pointer and is compatible with any pointer type. Dereferencing uses `^`:

```

TYPE
  NodePtr = POINTER TO Node;
VAR
  p : NodePtr;
BEGIN
  NEW(p);          (* allocate a Node on the heap *)
  p^.value := 42;
  DISPOSE(p);      (* free it - HeapFree; p is set to NIL *)
END;

```

The `Mod/tests/t-40-010-new-record.mod` tests this pattern. Every `NEW` must be paired with a `DISPOSE` before the pointer goes out of scope. See [Memory & exceptions](#).

Forward-declared pointer. A pointer may refer to a record type not yet declared in the same `TYPE` section. This is the only forward-reference allowed in Modula-2's `TYPE` block:

```

TYPE
  ListPtr = POINTER TO List;  (* List not yet visible - OK *)
  List    = RECORD
    value : INTEGER;
    next  : ListPtr;
  END;

```

AST node: `TypeExpr::Pointer(base, span)` (`ast.rs`, line 212). Sema defers resolution of `Unresolved` pointer targets until the full `TYPE` section has been processed.

Procedure types

A procedure type names the signature of a callable value - parameter modes and types, optional return type. Variables of procedure type hold function pointers:

TYPE

```
Comparator    = PROCEDURE(INTEGER, INTEGER) : BOOLEAN;  
Transform     = PROCEDURE(VAR INTEGER);  
EnumCallBack  = PROCEDURE(ARRAY OF CHAR);
```

`EnumCallBack` appears in `library/advapidef/ConfigSettings.def`. The Win32 bindings use procedure types with calling-convention attributes like `[EXPORT]` (`CUSTCNTL.def`, line 50: `LPFNCCSTYLEA = PROCEDURE(HWND, LPCCSTYLEA) : BOOL [EXPORT]`).

Procedure type variables are called like any other procedure:

```
VAR cmp : Comparator;  
BEGIN  
  cmp := MyLessThan;  
  IF cmp(a, b) THEN ... END;  
END;
```

AST node: `TypeExpr::Proc(ProcType)` where `ProcType` carries `params`, `return_ty`, and optional `attrs` (`ast.rs`, lines 256-261).

Opaque types

An opaque type is a `TYPE` name in a `DEFINITION MODULE` with no `=` body:

```
DEFINITION MODULE PipedExec;  
TYPE  
  PipedExecHandle;          (* opaque - clients see only the name *)
```

Clients import and use `PipedExecHandle` values - they can pass them to procedures and store them in variables - but cannot inspect or construct them. The structure lives entirely in the `IMPLEMENTATION MODULE`. NewM2 represents this as `TypeDecl { def: None, ... }` (`ast.rs`, line 127: `/// None indicates an opaque type`).

See [Modules & compilation](#) for how the definition and implementation module pair interact.

[NewM2 Guide home](#) ? [Lexical structure](#) ? [Expressions & operators](#) ? [Modules & compilation](#)

Expressions & Operators

Modula-2 has a compact, three-level binary expression grammar. Knowing the precedence table lets you write correct arithmetic and logical expressions without over-parenthesising.

Precedence

The table is read top-to-bottom: lower rows bind tighter. The parser implements exactly these levels (`src/newm2-parser/src/parser.rs`, functions `parse_relational` / `parse_simple` / `parse_term` / `parse_factor`).

Level	Operators	Associativity	AST function
Relational (lowest)	= # <> < <= > >= IN	non-associative	<code>parse_relational</code> (line 1691)
Additive / disjunctive	+ - OR	left	<code>parse_simple</code> (line 1718)
Multiplicative / conjunctive	* / DIV MOD REM AND & BOR BAND BXOR SHL SHR	left	<code>parse_term</code> (line 1756)
Prefix (highest)	NOT ~ BNOT unary + unary -	-	<code>parse_factor</code> / <code>parse_simple</code> (line 1828)

A parenthesised sub-expression (`e`) always resets to the outermost level (`parse_factor` line 1860).

Consequence: `AND` binds tighter than `OR`, which binds tighter than relational operators - so `a OR b AND c` parses as `a OR (b AND c)`, and `a = b OR c = d` parses as `(a = b) OR (c = d)`. You rarely need parentheses for the standard Boolean patterns.

Arithmetic and DIV / MOD / REM

The four standard arithmetic operators work on numeric types:

Operator	Meaning	Operand types
+	Addition	INTEGER, CARDINAL, REAL, LONGREAL, sets
-	Subtraction / negation	numeric, sets
*	Multiplication	numeric, sets
/	Real division, or set symmetric difference	REAL/LONGREAL, sets
DIV	Truncating integer division	integer types
MOD	Modulus - result has the sign of the divisor	integer types
REM	Remainder - result has the sign of the dividend	integer types (ISO)

The distinction between **MOD** and **REM** matters for negative operands:

```
VAR q, r : INTEGER;
BEGIN
  q := 7 DIV 2;    (* 3 - truncates toward zero *)
  r := 7 MOD 2;    (* 1 *)

  q := (-7) DIV 2; (* -3 - truncates toward zero *)
  r := (-7) MOD 2; (* 1 - MOD result has sign of divisor (2 is positive) *)
  r := (-7) REM 2; (* -1 - REM result has sign of dividend (-7) *)
END
```

This matches the corpus: [Mod/tests/t-10-020-divmod.mod](#) exercises **DIV** and **MOD** on positive integers. **REM** follows ISO 10514-1; NewM2 registers it as keyword **Rem** in [src/newm2-lexer/src/token.rs](#) line 136.

Unary **-** appears as a prefix on a factor:

```
VAR a, b : INTEGER;
BEGIN
  a := -5;
  b := 15 + a;  (* b = 10 *)
END
```

Source: [Mod/tests/t-10-030-neg-arith.mod](#).

Relational Operators and IN

Relational operators compare two values of the same base type and yield **BOOLEAN**:

Operator	Meaning	Note
=	Equal	
#	Not equal	
<>	Not equal	synonym for #
<	Less than	
<=	Less than or equal	
>	Greater than	
>=	Greater than or equal	
IN	Set membership	left: element type; right: set type

Both **#** and **<>** are distinct tokens (**TokenKind::Hash** and **TokenKind::NotEqual** in `src/newm2-lexer/src/token.rs` lines 72 and 76) but produce the same AST node **BinaryOp::Ne** (`ast.rs` line 329, `parser.rs` line 1697). Use whichever you prefer; the corpus uses both.

IN tests whether an ordinal value is a member of a set:

```
TYPE
  ExecFlags      = (ExecDetached, ExecMinimized, ExecMaximized, ExecHidden);
  ExecFlagSet    = SET OF ExecFlags;
VAR flags : ExecFlagSet;
BEGIN
  flags := ExecFlagSet{ExecMinimized, ExecHidden};
  IF ExecMinimized IN flags THEN
    (* minimized mode is set *)
  END;
END
```

Source pattern: `library/advapidef/PipedExec.def` defines `ExecFlagSet = SET OF ExecFlags`.

Relational expressions are non-associating in Modula-2: `a < b < c` is a parse error. Write `(a < b) AND (b < c)`.

Boolean Operators and Short-Circuit Evaluation

Modula-2 defines three Boolean operators:

Operator	Synonym	Meaning
AND	&	Logical conjunction - true iff both operands are true
OR	-	Logical disjunction - true iff at least one operand is true
NOT	~	Logical negation (prefix)

Both spellings of each operator are full keywords (`src/newm2-lexer/src/token.rs`: `Keyword::And`, `Keyword::Or`, `Keyword::Not`; `TokenKind::Amp`, `TokenKind::Tilde`). They produce the same AST nodes.

Short-circuit evaluation. `AND` and `OR` evaluate left-to-right and stop as soon as the result is determined:

- * `A AND B` - if `A` is `FALSE`, `B` is not evaluated.
- * `A OR B` - if `A` is `TRUE`, `B` is not evaluated.

This is guaranteed by PIM 4 and by the sequential recursive descent in `parse_simple` / `parse_term`. You may exploit it safely:

```
IF (p # NIL) AND (p^.value > 0) THEN
  (* safe: the deref only happens when p is non-nil *)
END;
```

`NOT` (or `~`) applies to a single factor:

```
IF NOT isPrime THEN
  divisor := candidate;
END;
```

Source: `Mod/tests/perf-primes-o2.mod` (line 24 uses `isPrime := FALSE`; the sieve at `perf-sieve-o2.mod` line 28 uses `flags[i] # 0` as a boolean guard).

Sets

Set operators use the same symbols as arithmetic operators, but apply to set types (**SET OF T** or **BITSET**). The type of both operands must be compatible set types.

Operator	Set operation
+	Union
-	Difference
*	Intersection
/	Symmetric difference

A **set constructor** builds a set value inline. The type name is written before the braces; elements and ranges are separated by commas:

```
TYPE
  Digit      = [0..9];
  DigitSet = SET OF Digit;
VAR s, t, u : DigitSet;
BEGIN
  s := DigitSet{1, 3, 5, 7, 9};    (* odd digits *)
  t := DigitSet{0, 2, 4, 6, 8};    (* even digits *)
  u := s + t;                      (* union - all digits *)
  s := u - DigitSet{0};            (* difference - remove 0 *)
  t := s * DigitSet{1..5};         (* intersection - {1, 2, 3, 4, 5} *)
END
```

A range inside a constructor is written **lo..hi**. The type-name prefix is optional for **BITSET** when the element type can be inferred, but writing it explicitly is recommended for readability.

Set constructor syntax in the AST: **Expr::Set { type_name, elements, span }** where each element is **SetElem::Single(e)** or **SetElem::Range(lo, hi)** (src/newm2-parser/src/ast.rs lines 297-298, 354-358; **parser.rs parse_set_constructor** line 2009).

INCL(s, e) and **EXCL(s, e)** are pervasive procedures that add and remove a single element; they are more efficient than **s := s + SetType{e}** for single-element updates. See [The standard environment](#).

Bitwise Operators (NewM2 Extension)

NewM2 carries these bitwise operators as reserved keywords (`src/newm2-lexer/src/token.rs` lines 153-159). They operate on integer types and have **no standard Modula-2 equivalent** - they are a NewM2 extension:

Operator	Operation	AST node
BAND	Bitwise AND	BinaryOp::Band
BOR	Bitwise OR	BinaryOp::Bor
BXOR	Bitwise exclusive OR	BinaryOp::Bxor
BNOT	Bitwise NOT (prefix)	UnaryOp::Not
SHL	Left shift	BinaryOp::Shl
SHR	Right shift	BinaryOp::Shr

BNOT is a prefix operator (parsed in `parse_factor`, line 1839); the rest are binary and sit at the multiplicative level in `parse_term` (lines 1769-1789) - the same level as **AND** and **DIV**.

The real corpus uses them heavily for multi-precision arithmetic:

```
(* From library/advapimod/Money.mod - multi-word integer arithmetic *)
a1[1] := a SHR DigitBits;
a3     := (a3 SHL 1) BOR (a2 SHR (DigitBits - 1));
RETURN (VAL(Money, a1) SHL DigitBits) BOR VAL(Money, a0);
```

Use these operators only in code that targets NewM2. Portable Modula-2 using the **SYSTEM** module's **CAST** and bit manipulation is the standard-conforming alternative.

Type Conversions

Modula-2 is strongly typed: arithmetic mixing of distinct numeric types requires an explicit conversion. The pervasive functions for this are:

Function	Converts
VAL(T, x)	Reinterprets x as type T (ordinal types; also numeric)
ORD(x)	Ordinal value of a CHAR, enumeration, or BOOLEAN
CHR(n)	CHAR with ordinal value n
FLOAT(n)	REAL from an integer
TRUNC(r)	INTEGER from REAL, truncating toward zero

VAL is the general-purpose conversion: VAL(INTEGER64, src) converts a LONGREAL to INTEGER64; VAL(INTEGER, mid) converts back. The corpus demonstrates a full round-trip in `Mod/tests/t-10-040-val-roundtrip.mod`.

These are pervasive identifiers - no import needed. They are described in full in [The standard environment](#).

[NewM2 Guide home](#) ? [Declarations & types](#) ? [Statements & control flow](#)

Statements & Control Flow

Every action in a Modula-2 body is a statement. Statements are **separated** by `;` (not terminated by it - a trailing `;` before `END` is legal because it introduces an empty statement). This page covers the full statement set that NewM2 parses, grounded in the `Stmt` enum in `src/newm2-parser/src/ast.rs`.

Assignment and calls

Assignment (`Stmt::Assign`) uses `:=`; the left side is any designator (variable, field, array element, or pointer dereference):

```
i      := 0;
arr[k] := arr[k] + 1;
p^.name := "hello";
```

Procedure call (`Stmt::Call`) is just a call expression used as a statement - no special syntax. The expression must ultimately resolve to a call:

```
STextIO.WriteLine;
SWholeIO.WriteInt(sum, 0);
INC(i);
```

The pervasive procedures `INC`, `DEC`, `INCL`, `EXCL` are identifiers (not keywords), so they appear here as ordinary call statements that sema resolves.

Note: `=` is **equality** (expression context only); `:=` is assignment. The parser (`Stmt::Assign`) rejects bare `=` in statement position, so this class of typo is caught early.

IF

`Stmt::If` holds a list of (`condition`, `body`) arms (covering both the `THEN` branch and any `ELSIF` branches) plus an optional `else_arm`:

```
PROCEDURE Category(n : INTEGER);
BEGIN
  IF n < 10 THEN
    STextIO.WriteString("low");
  ELSIF n < 100 THEN
    STextIO.WriteString("mid");
  ELSE
    STextIO.WriteString("high");
  END;
  STextIO.WriteLine;
END Category;
```

(From *Mod/tests/t-20-030-if-else.mod*)

ELSIF and **ELSE** are both optional; **ELSE** may appear at most once, after all **ELSIF** arms. The whole construct closes with **END**.

CASE

Stmt::Case matches an ordinal expression against a list of **CaseArm** values. Each arm is a **CaseLabel** list - either **CaseLabel::Single** (a single value) or **CaseLabel::Range** (an **a .. b** inclusive range). Arms are separated by **|**. An optional **ELSE** arm catches everything not matched; without it, a value that matches no label is a run-time error.

```
CASE RM OF
  rmNear: RETURN NearestLong(X);
| rmDown: RETURN FloorLong(X);
| rmUp:   RETURN CeilingLong(X);
| rmChop: RETURN TruncLong(X);
END;
```

(From *library/advapimod/Float.mod*, *ApplyRoundingMode*)

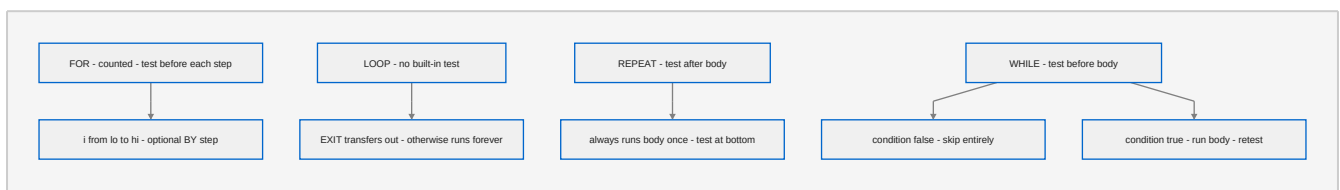
Multiple labels on one arm and ranges on one arm:

```
CASE code OF
  0, 1: result := "ok";
| 10 .. 19: result := "warning";
| 20 .. 99: result := "error";
ELSE    result := "unknown";
END;
```

The scrutinee must be an ordinal type: **INTEGER**, **CARDINAL**, **CHAR**, **BOOLEAN**, or an enumeration. **REAL** is not valid.

Loops

Modula-2 has four loop forms. They differ in where the exit condition sits, and whether iteration is counted.



WHILE

Stmt::While tests the condition **before** each iteration. If the condition is false on entry the body never runs:

```
i := 1;
WHILE i <= 5 DO
  SWholeIO.WriteInt(i, 0);
  STextIO.WriteLine;
  i := i + 1;
END;
```

(From *Mod/tests/t-20-010-while.mod*)

REPEAT

Stmt::Repeat tests the condition **after** the body, so the body always runs at least once. The loop continues while the condition is **FALSE** and exits when it becomes **TRUE**:

```
i := 0;
REPEAT
  SWholeIO.WriteInt(i, 0);
  STextIO.WriteLine;
  i := i + 1;
UNTIL i > 3;
```

(From *Mod/tests/t-20-040-repeat.mod*, outputs 0, 1, 2, 3)

FOR

Stmt::For counts a control variable from **start** to **end** inclusive. The optional **BY** clause (**step** field) defaults to 1. A negative step counts downward:

```
(* Sum 1..10 = 55 *)
sum := 0;
FOR i := 1 TO 10 DO
  sum := sum + i;
END;
```

(From *Mod/tests/t-20-020-for.mod*)

```
(* Iterate backwards - from library/win32mod/WinShell.mod *)
FOR i := 7 TO 0 BY -1 DO
  hexStr[i] := hexDig[num REM 16];
  num := num / 16;
END;
```

The control variable must be a local (or module-level) ordinal variable. Its value after the loop is

undefined by the standard; do not use it as if it were left at the last executed value.

LOOP and EXIT

`Stmt::Loop` introduces an unconditional loop with no built-in exit test. `Stmt::Exit` transfers control to the statement after the nearest enclosing `LOOP ... END`. This pair is Modula-2's general loop construct - use it when the exit condition falls naturally in the middle of the body:

```
LOOP
  IF i < Elements THEN
    IF idNum <> Cache^[i]^idNum THEN
      INC(i);
    ELSE
      (* found - promote to front, return *)
      RETURN TRUE;
    END;
  ELSE
    EXIT;    (* i >= Elements: not found *)
  END;
END;
RETURN FALSE;
```

(From library/win32mod/StringCache.mod, CacheHit)

`EXIT` is only meaningful inside a `LOOP` body; the parser accepts it anywhere in statement position (`Stmt::Exit`), but semantics requires an enclosing `LOOP`.

WITH

`Stmt::With` opens a record designator and makes its fields directly visible (unqualified) within the `DO ... END` body, just as if the fields were locally declared:

```
WITH Dialogs[dlgNum] DO
  FOR i := 0 TO numCtrls - 1 DO
    IF ctrls^[i].id = index THEN
      RETURN i;
    END;
  END;
END;
```

(From library/advapimod/DlgShell.mod, GetControlSubscript - Dialogs[dlgNum] is a record, and numCtrls, ctrls are its fields)

`WITH` reduces noise when accessing many fields of the same record. The designator is evaluated once on entry; the binding is purely lexical (not a pointer copy). Nesting is allowed; inner `WITH` fields shadow outer ones of the same name.

RETURN

`Stmt::Return` holds an optional expression (`Option<Expr>`):

- * In a **proper procedure** (no return type), `RETURN` with no expression exits early. Falling off the end is equivalent.
- * In a **function procedure** (has a return type), `RETURN expr` carries the result. Every control path must reach a `RETURN expr`; the standard requires it, and sema will enforce this.

```
(* Proper procedure - early exit *)
IF prc = NIL THEN
  RETURN;
END;

(* Function procedure *)
PROCEDURE SPLT(pos, w : INTEGER) : INTEGER;
BEGIN
  RETURN (pos - w DIV 2 + 1);
END SPLT;
```

(Pattern from library/win32mod/SplitterControl.mod)

Statement table

Statement	AST variant	Key keywords
Assignment	Stmt::Assign	:=
Procedure call	Stmt::Call	-
Empty	Stmt::Empty	bare ;
Conditional	Stmt::If	IF THEN ELSIF ELSE END
Case select	Stmt::Case	CASE OF ELSE END
Pre-test loop	Stmt::While	WHILE DO END
Post-test loop	Stmt::Repeat	REPEAT UNTIL
Counted loop	Stmt::For	FOR TO BY DO END
Unconditional loop	Stmt::Loop	LOOP END
Loop exit	Stmt::Exit	EXIT
Record open	Stmt::With	WITH DO END
Return value	Stmt::Return	RETURN
Raise exception	Stmt::Raise	RAISE
Exception retry	Stmt::Retry	RETRY
Exception block	Stmt::Block	EXCEPT FINALLY

The last three (**RAISE**, **RETRY**, **EXCEPT/FINALLY**) are ISO 10514-1 constructs - NewM2 parses them ([src/newm2-parser/src/ast.rs](#), `Stmt::Raise`, `Stmt::Retry`, `Stmt::Block`) but back-end execution is still being wired. See [Memory & exceptions](#).

[NewM2 Guide home](#) ? [Expressions & operators](#) ? [Procedures](#)

Procedures

A procedure is the fundamental unit of abstraction in Modula-2: named, typed, callable, and lexically scoped. This page covers declaring them, their parameters, nesting, and using them as values.

Proper procedures and function procedures

A **proper procedure** returns no value. It is called as a statement and its header ends after the parameter list:

```
PROCEDURE WriteValue(n : INTEGER);
BEGIN
  SWholeIO.WriteInt(n, 0);
  STextIO.WriteLine;
END WriteValue;
```

This is taken verbatim from `Mod/tests/T30020Helper.mod`. The closing `END` repeats the procedure name - the parser enforces this match (`src/newm2-parser/src/parser.rs`, procedure name-mismatch check).

A **function procedure** adds a result type after a colon. The body must execute a `RETURN` statement that produces a value of that type:

```
PROCEDURE BumpInt(n : INTEGER) : INTEGER;
BEGIN
  RETURN n + 4;
END BumpInt;
```

From `Mod/tests/T30030ScalarHelper.mod`, which also shows `PROCEDURE ... : CHAR` and `PROCEDURE ... : BOOLEAN`. The AST node is `ProcDecl` (`src/newm2-parser/src/ast.rs`, line 145): `return_ty: Option<TypeExpr> - None` for a proper procedure, `Some(...)` for a function procedure.

`RETURN` without an expression is valid in a proper procedure to exit early. In a function procedure, every path through the body must reach a `RETURN expr`.

Value and VAR parameters

Parameters are grouped between `(` and `)`. Parameters of the same type share a name list separated by commas:

```
PROCEDURE WriteRecordValues(leftIn : INTEGER; rightIn : CARDINAL; markIn : CHAR)
```

From `Mod/tests/T40020RecordHelper.mod`. Groups are separated by `;`.

Mode Syntax		Semantics
Value	name : T	The argument is copied; the procedure gets its own copy
VAR	VAR name : T	The procedure receives a reference to the caller's variable; writes to the parameter are visible in the caller

```
PROCEDURE Swap(VAR a, b : INTEGER);
VAR tmp : INTEGER;
BEGIN
  tmp := a;
  a := b;
  b := tmp;
END Swap;
```

A **VAR** parameter is declared with the **VAR** keyword before the name list; multiple names may share a single **VAR** prefix. The AST uses `ParamMode::Value` and `ParamMode::Var` (`src/newm2-parser/src/ast.rs`, lines 184-188). A **VAR** parameter test is exercised in the parser unit tests (`src/newm2-parser/src/lib.rs`, `procedure_with_var_param`).

Value semantics for structured types. Arrays and records passed by value are **copied** entirely - the callee cannot affect the caller's copy. For large arrays or records this copy is expensive; **VAR** is the standard way to pass them cheaply while signalling that modification is permitted. To pass read-only by reference, some Modula-2 dialects use a convention of **VAR** with a note that the procedure will not write; NewM2 also provides a **CONST** mode but that is an extension beyond what the standard tests.

Open array parameters

An **open array** formal accepts any concrete array of the right element type, regardless of length. The syntax is `ARRAY OF T` with no index bounds:

```
PROCEDURE Sum(a : ARRAY OF INTEGER) : INTEGER;
VAR i, s : INTEGER;
BEGIN
  s := 0;
  FOR i := 0 TO HIGH(a) DO
    s := s + a[i];
  END;
  RETURN s;
END Sum;
```

Inside the procedure, `HIGH(a)` returns the top index (length minus 1). The pervasive procedure `HIGH` is described in [The standard environment](#). The AST node for the formal's type is `TypeExpr::OpenArray(base, span)` (`src/newm2-parser/src/ast.rs`, line 209). Sema enforces that open-array types appear only in parameter position, not in **VAR** declarations or record fields.

The real library uses open arrays pervasively for string buffers:

```
PROCEDURE GetStringResource(idNum : CARDINAL; VAR str : ARRAY OF CHAR);
```

From `library/win32def/StringCache.def`. The `EnumCallback` callback type in `library/advapiodef/ConfigSettings.def` is also `PROCEDURE(ARRAY OF CHAR)`, meaning any procedure that takes an open char array fits this type.

An open array may itself be a `VAR` parameter (the `VAR` goes before `ARRAY OF T`):

```
PROCEDURE OpenConfig(companyName, appName : ARRAY OF CHAR;  
                    user : BOOLEAN) : CfgOpenStatus;
```

From `library/advapiodef/ConfigSettings.def`. Both `companyName` and `appName` are open char arrays passed by value; `user` is an ordinary value parameter of type `BOOLEAN`.

Nested procedures

A procedure body may declare local procedures in its declaration section (`ProcBody.decls` - `src/newm2-parser/src/ast.rs`, line 170-173). A nested procedure has access to all locals and parameters of every enclosing scope (static chain / upvalue capture):

```
PROCEDURE SortWords(VAR data : ARRAY OF INTEGER);  
  VAR n : INTEGER;  
  
  PROCEDURE Less(i, j : INTEGER) : BOOLEAN;  
  BEGIN  
    RETURN data[i] < data[j];    (* sees the enclosing data parameter *)  
  END Less;  
  
BEGIN  
  n := HIGH(data);  
  (* ... call Less ... *)  
END SortWords;
```

The nested `Less` sees `data` from the enclosing `SortWords` directly. Nesting is recursive - a nested procedure may itself contain further nested procedures. The AST represents this naturally: `ProcBody.decls` is a `Vec<Decl>`, and `Decl::Procedure` inside that vec is a nested procedure parsed identically to a top-level one (`src/newm2-parser/src/parser.rs`, `parse_top_decls`).

Status. Nested procedure scoping is resolved by sema. Lowering to LLVM IR for nested procedures that capture outer locals (closure upvalues) is a work in progress; nested procedures that do not capture outer variables lower correctly today.

FORWARD declarations

A **FORWARD** declaration gives the compiler a procedure's signature before the body appears. This lets two procedures call each other when one must be declared before the other:

```

PROCEDURE IsEven(n : INTEGER) : BOOLEAN; FORWARD;

PROCEDURE IsOdd(n : INTEGER) : BOOLEAN;
BEGIN
  IF n = 0 THEN RETURN FALSE END;
  RETURN IsEven(n - 1);
END IsOdd;

PROCEDURE IsEven(n : INTEGER) : BOOLEAN;
BEGIN
  IF n = 0 THEN RETURN TRUE END;
  RETURN IsOdd(n - 1);
END IsEven;

```

The **FORWARD** keyword follows the full header and a **;**; the body comes later in the same module. The parser sets `ProcDecl.is_forward = true` (`src/newm2-parser/src/ast.rs`, line 155) and `body = None` for the forward stub, then parses the full body when the second declaration appears.

DEFINITION modules as forward declarations. In practice, a procedure in a **DEFINITION MODULE** is purely a header - no body, parsed into a `ProcDecl` with `body: None` (`body: Option<ProcBody>`). The implementation module then supplies the body. This is the standard Modula-2 forward-declaration mechanism for exported procedures; explicit **FORWARD** inside a single module is needed only for mutual recursion within one file. See [Modules & compilation](#) for the definition/implementation pairing.

Procedure variables and procedure types

A *procedure type* describes the signature of a callable value. The [Declarations & types](#) page introduced the type form; here is the usage side.

A variable of procedure type holds a reference to any procedure with the matching signature:

```

TYPE
  Comparator = PROCEDURE(INTEGER, INTEGER) : BOOLEAN;

VAR
  cmp : Comparator;

PROCEDURE LessThan(a, b : INTEGER) : BOOLEAN;
BEGIN
  RETURN a < b;
END LessThan;

BEGIN
  cmp := LessThan;
  IF cmp(3, 7) THEN
    STextIO.WriteString("ordered");
  END;
END ...

```

The assignment `cmp := LessThan` stores a procedure reference. The call `cmp(3, 7)` calls it through the variable - useful for callbacks, dispatch tables, and functional idioms.

The predeclared type `PROC` is short for `PROCEDURE` with no parameters and no result, convenient for zero-argument callbacks:

```
VAR action : PROC;
BEGIN
  action := MyInit;
  action;      (* call with no arguments *)
END ...
```

The `Float.def` library uses procedure-type variables directly in `VAR` declarations:

```
VAR
  SetRoundingBoth : PROCEDURE (RoundingMode) [Pass(BX), Alters(BX)] = SetRounding87
  InitBoth        : PROCEDURE (          ) [          Alters( )] = Init87;
```

From `library/advapidef/Float.def`. The `[Pass(BX), Alters(BX)]` are register-passing attributes on the procedure type - a low-level extension for hand-optimised library code. The `EnumCallback = PROCEDURE (ARRAY OF CHAR)` type in `library/advapidef/ConfigSettings.def` shows a procedure type used as a callback parameter.

The AST represents both sides: a procedure type literal is `TypeExpr::Proc(ProcType)` and a procedure declaration is `Decl::Procedure(ProcDecl)`, where `ProcDecl.body` is `None` in a forward or definition-module header and `Some(ProcBody)` in a full declaration (`src/newm2-parser/src/ast.rs`, lines 144-173).

[NewM2 Guide home](#) ? [Statements & control flow](#) ? [Modules & compilation](#)

Objects & Classes

Modula-2's record-and-pointer core is enough for most data structures, but NewM2 also implements an **object-oriented layer built for COM** - the object model of the modern Windows API. It keeps the familiar Modula-2 OO *shape* (single-inheritance **classes** with virtual methods, **abstract** classes) but **deliberately departs from ISO 10514-3** where that standard's closed, GC-oriented model would get in the way: NewM2's design goal is to *fully support COM*, which is far more useful on Windows. The payoff is that an object's layout simply **is** the COM vtable ABI - **INTERFACE** types carry IIDs, and one M2 class can model your own hierarchy *and* consume a real OS COM object through the very same machinery. On top sits **runtime type discrimination** - **ISMEMBER** and the **GUARD** statement (new), constructs borrowed from the OO-Modula-2 tradition but lowered to serve this COM-unified model. The whole layer is *reference-based*.

This page is grounded in `src/newm2-sema/src/class.rs` (the class model), `analyze.rs` (checking), and `src/newm2-ir/src/lower.rs` + `src/newm2-runtime/src/rtti.rs` (layout + RTTI).

A class is a reference

A **class** is a record with methods, accessed only through a reference. An object variable holds a *pointer* to a heap object; assignment copies the reference, not the object.

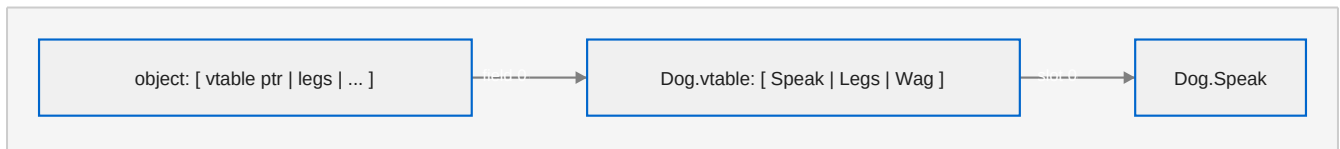
```
CLASS Animal;
  VAR legs : INTEGER;
  PROCEDURE Speak () : INTEGER;
  BEGIN RETURN 0 END Speak;
END Animal;

VAR a, b : Animal;
BEGIN
  NEW(a);          (* allocate an Animal on the heap *)
  a.legs := 4;
  b := a;           (* b and a now reference the SAME object *)
  b.legs := 2;      (* a.legs is now 2 as well *)
```

Fields and methods are reached with `.`; a method's body sees the object as the implicit **SELF**-like receiver (the fields are in scope unqualified). Classes are declared in an **IMPLEMENTATION**/program module; a **DEFINITION MODULE** may carry the class's *interface* (its public methods + fields), exactly mirroring the `.def/.mod` split for ordinary modules - see [Modules & compilation](#).

Object layout

Every object begins with a hidden **vtable pointer** at field 0, followed by the inherited fields then the class's own fields (`ClassSymbol.object_record`, `class.rs`). The vtable is a per-class array of method pointers; calling `a.Speak()` loads field 0, indexes the method slot, and calls it - so an overridden method dispatches on the object's *dynamic* class.



Single inheritance

A class may **INHERIT** at most one base. The derived class gets all of the base's fields and methods, and is a **subtype**: a **Dog** is assignable to an **Animal** variable (widening), but not the reverse.

```
CLASS Dog;
  INHERIT Animal;
  VAR tail : INTEGER;
  OVERRIDE PROCEDURE Speak () : INTEGER;    (* replace the inherited method *)
  BEGIN RETURN 2 END Speak;
  PROCEDURE Wag () : INTEGER;                (* a Dog-only method *)
  BEGIN RETURN tail END Wag;
END Dog;

VAR a : Animal; d : Dog;
BEGIN
  NEW(d);
  a := d;                                     (* OK: Dog is-a Animal - the STATIC type is Animal,
                                              the DYNAMIC type is Dog *)
  i := a.Speak();                             (* dispatches to Dog.Speak -> 2 (dynamic dispatch) *)
```

OVERRIDE replaces an inherited method while keeping its slot, so dispatch through a base reference reaches the most-derived implementation. A non-**OVERRIDE** method of the same name is an error - overriding is explicit.

Abstract classes

Prefix **ABSTRACT** to declare a class that cannot be instantiated, and **ABSTRACT** on a method to declare it without a body - a placeholder a concrete subclass must implement.

```
ABSTRACT CLASS Shape;
  ABSTRACT PROCEDURE Area () : INTEGER;    (* no body *)
END Shape;

CLASS Circle;
  INHERIT Shape;
  VAR r : INTEGER;
  OVERRIDE PROCEDURE Area () : INTEGER;
  BEGIN RETURN r * r * 3 END Area;
END Circle;
```

A concrete class that leaves an abstract method unimplemented is rejected (`class.rs::validate`). You may declare *variables* of an abstract type (they hold concrete subclass instances), but `NEW` on an abstract class is an error.

Object lifetime

Objects are heap-allocated and reference-counted by you, not the language:

- * **NEW(obj)** allocates the object and installs its vtable pointer at field 0.
- * **DISPOSE(obj)** frees it.
- * **EMPTY** is the null object reference - `EMPTY` is to objects as `NIL` is to pointers. A fresh object variable should be compared against `EMPTY`, and dereferencing an `EMPTY` reference faults.

```
VAR s : Shape;
BEGIN
  NEW(s, Circle);          (* dynamic type Circle, static type Shape *)
  IF s # EMPTY THEN i := s.Area() END;
  DISPOSE(s);
```

See [Memory & exceptions](#) for the allocator and the `--protect-heap / analyze` leak checks, which also cover object `NEW/DISPOSE`.

Interfaces & COM

An **INTERFACE** is a vtable-only class - no fields, all methods abstract - whose slot order *is* a COM interface's vtable. Annotated with its IID, it lets M2 call a real OS COM object directly: assign the raw interface pointer to an interface variable and dispatch through it.

```
INTERFACE IMalloc ["00000002-0000-0000-c000-000000000046"];
  ABSTRACT PROCEDURE QueryInterface () : INTEGER;      (* slot 0 *)
  ABSTRACT PROCEDURE AddRef () : INTEGER;              (* slot 1 *)
  ABSTRACT PROCEDURE Release () : INTEGER;             (* slot 2 *)
  ABSTRACT PROCEDURE Alloc (cb : CARDINAL) : ADDRESS;
  ABSTRACT PROCEDURE Free (p : ADDRESS);
END IMalloc;
```

Because NewM2's object layout *is* the COM ABI, an object's field-0 vtable pointer and an OS COM object's vtable are interchangeable: native method dispatch and COM method dispatch are the same instruction. The compiler's `@N` slot annotations turn "the generator transcribed the vtable" into "the build fails if a slot is off by one" - see [docs/design/com-interfaces.md](#). An interface may also be mirrored as an **ABSTRACT CLASS** (the historical idiom); both consume foreign COM objects.

Runtime type discrimination - ISMEMBER and GUARD

A base-class reference hides the object's dynamic type. Two constructs recover it **safely**, replacing the old hand-written "tag field + unchecked `CAST`" idiom.

ISMEMBER

```
ISMEMBER(p1, p2) : BOOLEAN
```

`ISMEMBER` is `TRUE` when `p1`'s class is `p2` or a subclass of it. Each operand is independently a class **type name** or an object **value**:

p1 \ p2	TYPE	VALUE
TYPE	compile-time constant fold	runtime, vs the value's dynamic type
VALUE	the common case (vs a fixed type)	runtime, vs another value's dynamic type

```
IF ISMEMBER(a, Dog) THEN ... END;      (* a's dynamic type is Dog or a Dog subclass
b := ISMEMBER(Dog, Animal);            (* folds to TRUE at compile time *)
```

A `NIL/EMPTY` object is a member of nothing, so `ISMEMBER(emptyVar, T)` is `FALSE`.

GUARD

`GUARD` is `CASE` over dynamic type: it tests the selector against each arm in order and runs the first that matches, **binding a read-only narrowed view** of the object in that arm.

```
PROCEDURE Describe (x : Shape);
BEGIN
  GUARD x AS
    c : Circle DO i := c.r          (* c is a Circle here - full Circle access
  | s : Square DO i := s.side
  ELSE
    (* x is some other Shape *)
  END
END Describe;
```

Semantics (`analyse_guard` in `analyze.rs`, `lower_guard` in `lower.rs`):

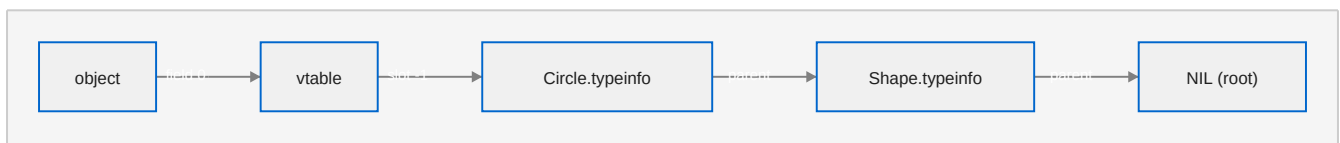
- * The selector is evaluated **once**.
- * Arms are tested **top-to-bottom; first match wins** - put specific types before a base-class catch-all.
- * The bound name (`C`, `S`) is a **read-only** narrowed view, scoped to that arm only; reading fields and calling methods through it needs no cast.
- * A `NIL/EMPTY` selector takes the `ELSE`

- * With **no matching arm and no ELSE**, **GUARD** raises **guardException** - a NewM2 exception with its own source (it is deliberately *not* one of the closed ISO **M2EXCEPTION** codes), catchable in an **EXCEPT** handler.
- * **GUARD**, **AS**, and **ISMEMBER** are **soft keywords**: existing identifiers spelled **guard** / **as** / **ismember** keep working.

The arm types must be related to the selector's class (an unrelated arm is a compile error), turning the old "the tag and the cast disagreed" runtime bug into a build failure.

How it works - RTTI at vtable[-1]

Each native class emits a constant `{Class}.typeinfo` descriptor - `{ parent, name, depth }` (`src/newm2-runtime/src/rtti.rs`) - for **every** class, abstract bases included, so the ancestor chain is always complete. The pointer to it sits **one slot before** the object's vtable (`vtable[-1]`), so ordinary method dispatch is byte-for-byte unchanged (and identical for foreign COM objects, which simply have no such slot). **ISMEMBER/GUARD** read the object's typeinfo and walk the parent chain (`nm2_rtti_isa`, with a `depth` early-out):



`nm2_rtti_isa(cand, target)` returns TRUE iff `target` is found while walking `cand`'s parent chain - i.e. `cand` is `target` or a subclass.

What's live, what's pending

- * **Live:** classes, single inheritance, abstract classes + **OVERRIDE**, **NEW/DISPOSE/EMPTY**, virtual dispatch, **INTERFACE** consumption of foreign COM, and **native-class ISMEMBER + GUARD** (the design is `docs/design/guard-ismember.md`).
- * **Pending:** **GUARD/ISMEMBER** on an **interface** (it needs a `QueryInterface` probe, not yet built) is rejected at compile time rather than silently misbehaving.
- * **Deliberate departures from ISO 10514-3** (the COM model wins): no **TRACED** (garbage-collected) classes - lifetime is manual / COM-refcounted; no operator overloading; no universal root object.

See also *Declarations & types*, *Memory & exceptions*, and the design note `docs/design/guard-ismember.md`.

The Standard Environment

Everything available to a Modula-2 program without any `IMPORT` - the pervasive procedures and functions, the `SYSTEM` pseudo-module, and the key library modules you reach with a single `IMPORT` line.

Pervasive procedures and functions

The names below are *pervasive identifiers*: predeclared in an enclosing pseudo-scope, visible in every module without import, and re-declarable (though you should not). The lexer returns them as plain `Ident` tokens; sema resolves them. The complete registration is at `src/newm2-sema/src/analyze.rs` line 471 - every name in the `for name in [...]` loop is confirmed present.

Storage

Procedure	Signature	What it does
NEW	NEW(p) or NEW(p, n)	Allocates a record (or n elements) on the heap via HeapAlloc and assigns the pointer p. The payload is zero-initialised. Must be paired with DISPOSE.
DISPOSE	DISPOSE(p)	Frees the heap object pointed to by p via HeapFree and sets p to NIL. Required to avoid leaks.
HALT	HALT	Terminates the program immediately.
ASSERT	ASSERT(b) or ASSERT(b, n)	Halts with an optional error code if b is FALSE.

`NEW` and `DISPOSE` are covered in depth in [Memory & exceptions](#).

Ordinal

Function	Signature	What it does
INC	INC(v) or INC(v, n)	Increments an ordinal variable v by 1 or by n.
DEC	DEC(v) or DEC(v, n)	Decrements an ordinal variable v by 1 or by n.
ODD	ODD(i) : BOOLEAN	Returns TRUE if the integer i is odd.

`INC` and `DEC` are *procedures*, not functions: they modify their first argument in place. The optional second argument must be a constant or variable of any integer type.

Set

Procedure	Signature	What it does
INCL	INCL(s, e)	Includes element e in set s (equivalent to $s := s + \{e\}$).
EXCL	EXCL(s, e)	Excludes element e from set s (equivalent to $s := s - \{e\}$).

```
VAR flags : BITSET;  
BEGIN  
  flags := BITSET{};  
  INCL(flags, 3);    (* flags = {3} *)  
  EXCL(flags, 3);    (* flags = {} *)  
END;
```

Array

Function	Signature	What it does
HIGH	HIGH(a) : CARDINAL	Returns the upper bound of the first (or only) open-array dimension. For a formal ARRAY OF T parameter, HIGH(a) is the index of the last element.

```
PROCEDURE Sum(a : ARRAY OF INTEGER) : INTEGER;  
VAR i, s : INTEGER;  
BEGIN  
  s := 0;  
  FOR i := 0 TO HIGH(a) DO s := s + a[i] END;  
  RETURN s  
END Sum;
```

Size

Function	Signature	What it does
SIZE	SIZE(v) : CARDINAL	Storage size of a variable or a type expression, in bytes. SIZE(INTEGER) and SIZE(x) both work.
TSIZE	TSIZE(T) : CARDINAL	Size of a type, always taking a type expression (not a variable). Identical to SIZE when given a type; TSIZE is the ISO-preferred spelling for the type-only form.

Both **SIZE** and **TSIZE** are registered in the pervasive scope ([src/newm2-sema/src/analyze.rs](#) line 471). In constant expressions **SIZE(T)** is evaluated at compile time.

Conversion and coercion

Function	Signature	What it does
ORD	ORD(x) : CARDINAL	Ordinal value of a CHAR, BOOLEAN, or enumeration value.
CHR	CHR(n) : CHAR	Character whose ordinal is n.
VAL	VAL(T, x)	Converts value x to type T. The first argument is a type name. Used for integer width changes, ordinal-to-enumeration, and scalar cross-family conversions.
CAP	CAP(c) : CHAR	Uppercase of CHAR c; letters become uppercase, others are unchanged.
ODD	ODD(i) : BOOLEAN	(See Ordinal above.)
ABS	ABS(x) : T	Absolute value of an integer or real. Returns the same type as its argument.
FLOAT	FLOAT(i) : REAL	Converts an integer to REAL.
TRUNC	TRUNC(r) : INTEGER	Truncates a REAL or LONGREAL to INTEGER (towards zero).
INT	INT(x) : INTEGER	Converts a cardinal or other ordinal to INTEGER. Registered in sema (analyze.rs line 474).
LFLOAT	LFLOAT(x) : LONGREAL	Converts an integer or REAL to LONGREAL. Registered in sema.
ENTIER	ENTIER(r) : INTEGER	Floor of a real value (largest integer not greater than r). ISO 10514-1. Registered in sema.
MIN	MIN(T)	Minimum value of an ordinal or real type. Used in constant expressions.
MAX	MAX(T)	Maximum value of an ordinal or real type. Used in constant expressions.

A round-trip example from [Mod/tests/t-10-040-val-roundtrip.mod](#):

```
VAR src : LONGREAL; mid : INTEGER64;  
BEGIN  
  src := 42.0;  
  mid := VAL(INTEGER64, src);  
  SwholeIO.WriteInt(VAL(INTEGER, mid), 0);  
END
```

Not registered: Standard PIM also lists `FLOAT` as accepting a `LONGREAL` argument for explicit widening; NewM2 registers it as a single pervasive name and relies on the type checker to handle both widths. The ISO `FLOATD` (double) alias is not separately registered - use `LFLOAT` or `VAL(LONGREAL, x)` instead.

The SYSTEM module

`SYSTEM` is a *pseudo-module* built directly into sema - it has no `.def` file. Import it with `IMPORT SYSTEM;` and use names qualified (`SYSTEM.ADDRESS`, `SYSTEM.CAST`, ...). The full registration is in `src/newm2-sema/src/analyze.rs` starting at line 496 (`build_intrinsic_module_scope` for "SYSTEM").

Types exported from SYSTEM

Name	What it is
ADDRESS	Pointer-sized unsigned integer. The canonical type for raw pointer arithmetic (SYSTEM.ADDRESS and the pervasive ADDRESS name both map to Builtin::SysAddress).
WORD	Native machine word (pointer-width unsigned).
BYTE	Single-byte storage unit (Builtin::SysByte).
LOC	Smallest addressable unit (Builtin::SysLoc). ISO 10514-1.
BITSET	SYSTEM.BITSET - same width as a machine word; available qualified to avoid ambiguity with the pervasive BITSET.
ADRINT	Signed address-arithmetic integer.
ADRCARD	Unsigned address-arithmetic integer (MACHINEWORD is a registered alias).
CARD8/16/32/64	Exact-width cardinal aliases for C interop.
INT8/16/32/64	Exact-width signed aliases for C interop.
PROTECTION	Interrupt protection level (cardinal-sized).
VA_LIST	C variadic argument list pointer (address-sized).
FUNC	Generic function/procedure pointer (address-sized).

Procedures exported from SYSTEM

Name	What it does
ADR(v) : ADDRESS	Address of variable v. The fundamental address-of intrinsic.
CAST(T, x)	Type transfer: reinterprets the bit pattern of x as type T with no conversion. SYSTEM.CAST(INTEGER64, src) - see Mod/tests/t-10-050-system-cast.mod.
TSIZE(T) : CARDINAL	Type size (also available pervasively).
ADDADR SUBADR DIFADR MAKEADR	Address arithmetic (add/subtract/difference/construct).
SHIFT(x, n)	Logical bit shift; positive n shifts left.
ROTATE(x, n)	Bit rotation.
OFFS(p, n)	Offset-from-pointer arithmetic (NewM2 extension).
SWAPENDIAN BIGENDIAN	Endian-swap intrinsics (NewM2 extension).
PUSHREGISTERS POPREGISTERS	Save/restore callee registers around low-level asm stubs.
VA_START VA_END VA_ARG	C variadic argument access.
UNREFERENCED_PARAMETER	Suppress unused-variable warnings (NewM2 extension).

```

IMPORT SYSTEM;
VAR src : INTEGER; wide : INTEGER64;
BEGIN
  src := 17;
  wide := SYSTEM.CAST(INTEGER64, src);
END

```

Coroutines - SYSTEM vs COROUTINES

The classic PIM **SYSTEM** exports (**NEWPROCESS**, **TRANSFER**, **IOTRANSFER**, **GetCurrentCoroutineId**) are registered in sema as *legacy aliases* ([analyze.rs](#) line 544), but calling them produces a focused "not yet implemented" diagnostic pointing to the ISO **COROUTINES** pseudo-module.

The ISO **COROUTINES** pseudo-module (also intrinsic) exports: **COROUTINE** (a type), **INTERRUPTSOURCE**, **NEWCOROUTINE**, **TRANSFER**, **IOTRANSFER**, **ATTACH**, **DETACH**, **IsATTACHED**, **HANDLER**, **CURRENT**, **LISTEN**

, `PROT`, `COROUTINEDONE`. These names are registered in `sema` (lines 577-629) but coroutine execution is **deferred** - the scaffolding is in place so name resolution does not fail, but the back-end does not yet generate working coroutine code.

Key library modules

These modules are resolved at compile time as ordinary `.def/.mod` pairs or as JIT runtime shims; import them with a plain `IMPORT` statement.

Text I/O: `STextIO` and `SWholeIO`

`STextIO` is the primary character-output module. The JIT binds its procedures directly to runtime helpers in `src/newm2-llvm/src/lib.rs` (line 242):

Procedure	What it does
<code>STextIO.WriteString(s)</code>	Write a string to standard output.
<code>STextIO.WriteLine</code>	Write a newline.
<code>STextIO.WriteChar(c)</code>	Write a single CHAR.
<code>SWholeIO.WriteInt(i, w)</code>	Write signed integer <code>i</code> in field width <code>w</code> .
<code>SWholeIO.WriteCard(c, w)</code>	Write cardinal <code>c</code> in field width <code>w</code> .

The classic hello-world (from `Mod/tests/t-30-010-write-str.mod`):

```
MODULE Hello;
IMPORT STextIO;
BEGIN
  STextIO.WriteString("Hello, NewM2!");
  STextIO.WriteLine;
END Hello.
```

`InOut` is a PIM-classic alias: `InOut.WriteString`, `InOut.WriteLine`, `InOut.Write`, `InOut.WriteInt`, and `InOut.WriteCard` are all bound to the same runtime shims.

SRealIO and SLongIO. The ISO standard also defines `SRealIO` (real I/O) and `SLongIO` (long-integer I/O). These are not yet wired in the JIT runtime - use `SWholeIO.WriteInt` for integers and format reals via `VAL` conversion for now.

Storage

Storage provides the ISO-standard heap allocation interface that underlies **NEW** and **DISPOSE**:

Procedure	What it does
Storage.ALLOCATE(p, n)	Allocate n bytes; assign the ADDRESS to p.
Storage.DEALLOCATE(p, n)	Free n bytes starting at p.

ALLOCATE calls **HeapAlloc**; **DEALLOCATE** calls **HeapFree**. You rarely call **Storage** directly - use **NEW**/**DISPOSE** instead.

Float

Float is a library module providing floating-point exception management and rounding conversions. From `library/advapidef/Float.def` (exercised in `Mod/tests/t-50-070-float-library.mod`):

```
IMPORT Float, STextIO, SWholeIO;
VAR flags : Float.FPExceptions;
BEGIN
  Float.Init;
  Float.ClearFPExceptions;
  SWholeIO.WriteInt(Float.NearestToInt32(1.75), 0);
  STextIO.WriteLine;
  flags := Float.GetFPExceptions() + Float.FPException{Float.exOverflow};
  IF Float.exOverflow IN flags THEN ... END;
END
```

Float.FPExceptions is a **PACKEDSET OF FPException**, and set-constructor syntax **Float.FPException{...}** is standard Modula-2.

SYSTEM (library view)

As described above, **SYSTEM** is pseudo-module with no **.def** file - **IMPORT SYSTEM;** is always available and never requires a library path.

The NM2.IO Unicode extensions

For **UCHAR/ACHAR** text output, NewM2 provides a non-standard **NM2.IO** qualified namespace bound at JIT time (`src/newm2-llvm/src/lib.rs` lines 252-255): **NM2.IO.WriteUChar**, **NM2.IO.WriteUInt**, **NM2.IO.WriteUCard**, **NM2.IO.WriteString**. These are NewM2 extensions; they do not appear in ISO Modula-2.

Memory & Exceptions

How NewM2 allocates heap objects, reclaims them, and handles runtime exceptions - two topics that share a common thread: what happens when things go wrong at runtime.

Pointers

POINTER TO T is a heap pointer type. A variable of pointer type holds either a heap address or the null value **NIL**. **NIL** is assignment-compatible with every pointer type.

```
TYPE
  Node      = RECORD
    value   : INTEGER;
    serial  : INTEGER;
  END;
  NodePtr = POINTER TO Node;

VAR p : NodePtr;
```

AST node: `TypeExpr::Pointer(Box<TypeExpr>, Span)` (`src/newm2-parser/src/ast.rs`, line 213).

Dereferencing uses the postfix [^] operator. `p^` yields the **Node** that `p` points to. Fields are then accessed with `·`:

```
p^.value := 99;
p^.serial := p^.value * 2;
```

This exact pattern is in `Mod/tests/t-40-010-new-record.mod` - the test allocates a **Node** record, writes both fields, and reads them back.

Forward-declared pointers. A **POINTER TO T** may appear in a **TYPE** section before **T** is defined, as long as both appear in the same **TYPE** block. This is the standard Modula-2 idiom for linked structures:

```
TYPE
  ListPtr = POINTER TO List;  (* List not yet declared - OK *)
  List    = RECORD
    value : INTEGER;
    next  : ListPtr;
  END;
```

Sema defers resolution of the **Unresolved** pointer target until the full **TYPE** section has been processed.

NEW and DISPOSE

NEW and **DISPOSE** are pervasive procedures - predeclared identifiers, not reserved words. The lexer returns them as ordinary **Ident** tokens; sema resolves them.

- * **NEW(p)** allocates a heap record whose type is the base type of **p** and assigns the pointer. The payload is zero-initialised. The runtime entry point calls **Storage.ALLOCATE**, which maps to **HeapAlloc**.
- * **DISPOSE(p)** frees the object and sets **p** to **NIL**. It calls **Storage.DEALLOCATE**, which maps to **HeapFree**. Every **NEW** must be paired with a **DISPOSE** before the pointer goes out of scope to avoid leaks.

A minimal working example (from `Mod/tests/t-40-010-new-record.mod`):

```
MODULE T40010NewRecord;
IMPORT STextIO, SWholeIO;

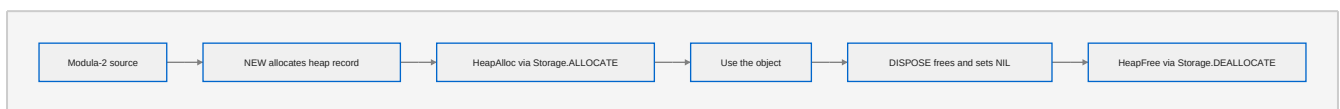
TYPE
  Node    = RECORD value : INTEGER; serial : INTEGER; END;
  NodePtr = POINTER TO Node;

VAR p : NodePtr;

BEGIN
  NEW(p);
  p^.value := 99;
  p^.serial := p^.value * 2;
  SWholeIO.WriteInt(p^.value, 0);
  STextIO.WriteLine;
  SWholeIO.WriteInt(p^.serial, 0);
  STextIO.WriteLine;
  DISPOSE(p);
END T40010NewRecord.
```

Manual memory management

NewM2 uses classical manual memory: **Storage.ALLOCATE** maps to **HeapAlloc** and **Storage.DEALLOCATE** maps to **HeapFree**. Every **NEW** allocation must be paired with a **DISPOSE** before the pointer goes out of scope.




```

VAR p : NodePtr;
BEGIN
    NEW(p);
    (* ... use p ... *)
    DISPOSE(p);    (* frees the block; p is set to NIL *)
END;

```

There is no collector, no safepoint overhead, and no conservative stack scan.

Exceptions: EXCEPT, FINALLY, RETRY

ISO 10514-1 extends Modula-2's statement syntax with structured exception handling. The exception mechanism wraps a body of statements in a *protected block* that can catch and re-run on error. NewM2 **parses this syntax fully** (confirmed in [src/newm2-parser/src/ast.rs](#), lines 361-411) but **back-end execution of exception handling is deferred** - RAISE compiles to [llvm.trap](#) and RETRY is a no-op in the current lowering phase ([src/newm2-ir/src/lower.rs](#), lines 772-779; [src/newm2-llvm/src/codegen.rs](#), line 1008).

Document the syntax and semantics now; they are what ISO programs will use once the back-end wires up the EH landingpads (currently planned for Phase 8 per the [lower.rs](#) comment at line 778).

Protected block structure

Any module body or procedure body may have an **EXCEPT** part and/or a **FINALLY** part appended before the closing **END**. The AST carries these as fields of **Block**: **except**: [Vec<ExceptArm>](#) and **finally**: [Option<Vec<Stmt>>](#) ([ast.rs](#), lines 363-366).

```

BEGIN
    (* protected body *)
    DoSomethingRisky;
    OpenFile(name, f);
EXCEPT
    FileNotFound : STextIO.WriteString("file not found"); |
    AccessDenied : STextIO.WriteString("access denied");
FINALLY
    CloseFile(f);
END;

```

- * The **EXCEPT** part lists one or more *exception arms*. Each arm names one or more exception identifiers ([ExceptArm.names](#): [Vec<QualName>](#)), followed by a colon and a statement sequence. Arms are separated by **|**.
- * A bare **EXCEPT** with no names is a catch-all (an [ExceptArm](#) with an empty [names](#) vec). This catches any exception not matched by an earlier named arm.
- * The **FINALLY** part is a statement sequence that runs whether or not an exception was raised - always executed on exit from the protected block, normal or exceptional. Use it for cleanup such as releasing resources.

RAISE

RAISE raises an exception. It appears as `Stmt::Raise(Option<Expr>, Span)` in the AST (`ast.rs`, line 407). With an expression it raises a specific exception value; bare **RAISE** (inside an exception handler) re-raises the current exception:

```
RAISE FileNotFound;      (* raise a specific exception *)
RAISE;                   (* re-raise; only inside an EXCEPT arm *)
```

Current status: the lowering phase emits `llvm.trap` for **RAISE**, so any module that executes a **RAISE** statement will trap immediately. This is correct behaviour for catching the case at test time; structured dispatch is Phase 8.

RETRY

RETRY is a statement that re-runs the protected body from the beginning. It may only appear inside an **EXCEPT** arm.

```
VAR attempts : INTEGER;
BEGIN
  attempts := 0;
EXCEPT
  TransientError :
    INC(attempts);
    IF attempts < 3 THEN RETRY END;
END;
```

Current status: the lowering phase silently drops **RETRY** (`lower.rs`, line 777: "Phase 4: **RETRY** is a structural no-op; full EH in Phase 8"). Code that depends on re-execution will not retry in the current build; it will fall through as if **RETRY** were an empty statement.

Anonymous exception blocks

An **EXCEPT**/**FINALLY** block may appear as a statement in its own right - not only at the end of a procedure body. The parser represents this as `Stmt::Block(Block)` where the `Block` carries its own `except` and `finally` fields (`ast.rs`, lines 409-411). Nesting is supported.

```
PROCEDURE Transfer(src, dst : ARRAY OF CHAR);
BEGIN
  (* outer body *)
  BEGIN
    CopyFile(src, dst);
  EXCEPT
    IOError : HandleCopyError(src, dst);
  FINALLY
    ReleaseLock;
  END;
```

```
(* outer body continues *)  
END Transfer;
```

Summary of exception status

Construct	Parses	IR lowered	Back-end executes
EXCEPT arms	Yes	Structural skeleton	Deferred Phase 8
FINALLY block	Yes	Structural skeleton	Deferred Phase 8
RAISE expr	Yes	Emits llvm.trap	Traps immediately
RETRY	Yes	No-op (dropped)	Silently skipped

Write exception-guarded code today if you are building for the future; just be aware that only **RAISE** produces observable behaviour (a trap), while **EXCEPT** handlers and **RETRY** are not yet dispatched.

[NewM2 Guide home](#) ? [Declarations & types](#) ? [The standard environment](#)

Reference

The reserved-word set and the predeclared (pervasive) identifiers. The reserved words are from `src/newm2-lexer/src/token.rs`; the pervasive identifiers are registered by sema (`src/newm2-sema/src/analyze.rs`).

Reserved words

These are the keywords the lexer recognises - PIM 4 + ISO 10514-1, a piece of ISO 10514-2, and NewM2 extensions. They cannot be used as identifiers.

Group	Reserved words
Modules & structure	MODULE DEFINITION IMPLEMENTATION IMPORT FROM EXPORT QUALIFIED BEGIN END
Declarations	CONST TYPE VAR PROCEDURE FORWARD RECORD ARRAY SET POINTER OF TO
Control flow	IF THEN ELSIF ELSE CASE WHILE DO REPEAT UNTIL FOR BY LOOP EXIT WITH RETURN
Word operators	AND OR NOT DIV MOD REM IN
ISO 10514-1	EXCEPT FINALLY RETRY GENERIC PACKEDSET
Object-oriented (COM)	ABSTRACT
NewM2 extensions	ASM BAND BOR BXOR BNOT SHL SHR

Contextual (soft) keywords. The object-oriented layer's other words - `CLASS` `INTERFACE` `INHERIT` `REVEAL` `OVERRIDE` `GUARD` `AS` (and `UNSAFEGUARDED`) - are recognised only in their syntactic position, so they are *not* reserved and remain usable as ordinary identifiers. See [Objects & classes](#).

Pervasive identifiers

Predeclared in an enclosing pseudo-scope and usable without an import - but they are ordinary **identifiers**, not reserved words (so a local declaration shadows them).

Types - `INTEGER` `CARDINAL` `REAL` `LONGREAL` `BOOLEAN` `CHAR` `BITSET` `PROC`, the ISO `COMPLEX` / `LONGCOMPLEX`, and the exact-width family `INTEGER8/16/32/64`, `CARDINAL8/16/32/64`, `ACHAR`, `UCHAR`.

Constants - `TRUE` `FALSE` `NIL`, and `EMPTY` (the null object reference - `EMPTY` is to objects as `NIL` is to pointers; see [Objects & classes](#)).

Procedures & functions (registered in `analyze.rs`) - see [The standard environment](#) for what each does:

Kind	Names
Storage	NEW DISPOSE
Increment / set	INC DEC INCL EXCL
Inquiry	HIGH SIZE TSIZE ODD
Conversion	ORD CHR VAL CAP FLOAT LFLOAT INT TRUNC ENTIER
Arithmetic	ABS MIN MAX
Object RTTI	ISMEMBER (and the GUARD statement - see Objects & classes)
Program	HALT ASSERT

The SYSTEM module

`IMPORT SYSTEM`; exposes the low-level layer: types `ADDRESS WORD BYTE LOC` (and `CARD8..64 / INT8..64`), and procedures `ADR` (address-of), `CAST / VAL` (type transfer), `TSIZE`, address arithmetic `ADDADR / SUBADR / DIFADR / MAKEADR`, and `SHIFT / ROTATE`. The coroutine primitives (`NEWPROCESS / TRANSFER / IOTRANSFER`) parse and resolve but are not yet executed. See [The standard environment](#).

Operators at a glance

`:=` assign ? `=` equal ? `# / <>` not-equal ? `< <= > >=` order ? `IN` membership ? `+ - * /` arithmetic and set ops ? `DIV MOD REM` integer division ? `AND / & OR NOT / ~` boolean ? NewM2 `BAND BOR BXOR BNOT SHL SHR` bitwise. Full precedence on [Expressions & operators](#).

[NewM2 Guide home](#) ? [Lexical structure](#) ? [The standard environment](#)

Grammar & Syntax Summary

A compact EBNF of the language **as NewM2 parses it** ([src/newm2-parser/src/parser.rs](#)).

Notation: `[x]` optional, `{x}` zero-or-more, `a | b` alternatives, `"x"` a literal token. This is a reading aid, not the normative grammar; see the chapter pages for semantics.

Compilation units

A program is a **MODULE**; a library is a **DEFINITION** interface plus an **IMPLEMENTATION** body.

```
CompilationUnit = ProgramModule | DefinitionModule | ImplementationModule .

ProgramModule      = "MODULE" ident [ Priority ] ";" { Import } Block ident "." .
DefinitionModule   = "DEFINITION" "MODULE" ident ";" { Import } { Definition } "END" ident .
ImplementationModule = "IMPLEMENTATION" "MODULE" ident [ Priority ] ";" { Import } Block ident "." .

Import = [ "FROM" ident ] "IMPORT" identList ";" .
Block  = { Declaration } [ "BEGIN" StatementSeq ] [ Protection ] "END" .
```

Declarations

```
Declaration = "CONST" { ConstDecl ";" }
              | "TYPE"  { TypeDecl  ";" }
              | "VAR"   { VarDecl   ";" }
              | ProcedureDecl ";"
              | ModuleDecl ";"
              | ClassDecl ";" .      (* see "Classes & interfaces" below *)

ConstDecl = ident "=" ConstExpr .
TypeDecl  = ident "=" Type .
VarDecl   = identList ":" Type .

ProcedureDecl = "PROCEDURE" ident [ FormalParams ] ";" Block ident
                | "PROCEDURE" ident [ FormalParams ] ";" "ASM" ... "END" ident . (* inline *)

FormalParams = "(" [ FPSection { ";" FPSection } ] ")" [ ":" qualident ] .
FPSection    = [ "VAR" ] identList ":" FormalType .
```

Types

```

Type = qualident
    | "ARRAY" Type { "," Type } "OF" Type
    | "RECORD" FieldList "END"
    | "SET" "OF" Type                | "PACKEDSET" "OF" Type
    | "POINTER" "TO" Type
    | "PROCEDURE" [ FormalTypeList ]
    | "(" identList ")"              (* enumeration *)
    | ConstExpr ".." ConstExpr      (* subrange *) .

FieldList = Field { ";" Field } [ "CASE" [ ident ":" ] qualident "OF" Variant { "|" Variant

```

Built-in scalar types and the exact-width family are listed in [Reference](#).

Expressions

```

Expr      = SimpleExpr [ Relation SimpleExpr ] .
Relation  = "=" | "#" | "<>" | "<" | "<=" | ">" | ">=" | "IN" .
SimpleExpr = [ "+" | "-" ] Term { ("+" | "-" | "OR") Term } .
Term       = Factor { ("*" | "/" | "DIV" | "MOD" | "REM" | "AND" | "&") Factor } .
Factor     = Designator [ ActualParams ]
            | Number | String | Set | "(" Expr ")" | ("NOT" | "~") Factor .
Designator = qualident { "." ident | "[" Expr { "," Expr } "]" | "^" } .

```

Bitwise word operators are NewM2 extensions: **BAND BOR BXOR BNOT SHL SHR**.

Statements

```

StatementSeq = Statement { ";" Statement } .
Statement = [ Assignment | ProcCall
            | IfStmt | CaseStmt | GuardStmt
            | WhileStmt | RepeatStmt | ForStmt | LoopStmt | WithStmt
            | "EXIT" | "RETURN" [ Expr ] | "RAISE" [ Expr ] | "RETRY" ] .

Assignment = Designator ":=" Expr .
IfStmt     = "IF" Expr "THEN" StatementSeq { "ELSIF" Expr "THEN" StatementSeq } [ "ELSE" StatementSeq ] "END" .
CaseStmt   = "CASE" Expr "OF" Case { "|" Case } [ "ELSE" StatementSeq ] "END" .
ForStmt    = "FOR" ident ":=" Expr "TO" Expr [ "BY" ConstExpr ] "DO" StatementSeq "END" .
LoopStmt   = "LOOP" StatementSeq "END" .    (* leave with EXIT *)

```

A protected region uses ISO exception handling:

```

Protection = "EXCEPT" StatementSeq | "FINALLY" StatementSeq [ "EXCEPT" StatementSeq ] .

```

Classes & interfaces (the COM-native object layer)

```
ClassDecl = [ "ABSTRACT" ] "CLASS" ident ";" [ "INHERIT" qualident ";" ]
           { ClassMember } "END" ident .
Interface = "INTERFACE" ident [ "[" iidString "]" ] ";" [ "INHERIT" qualident ";" ]
           { "ABSTRACT" MethodHeader ";" } "END" ident .

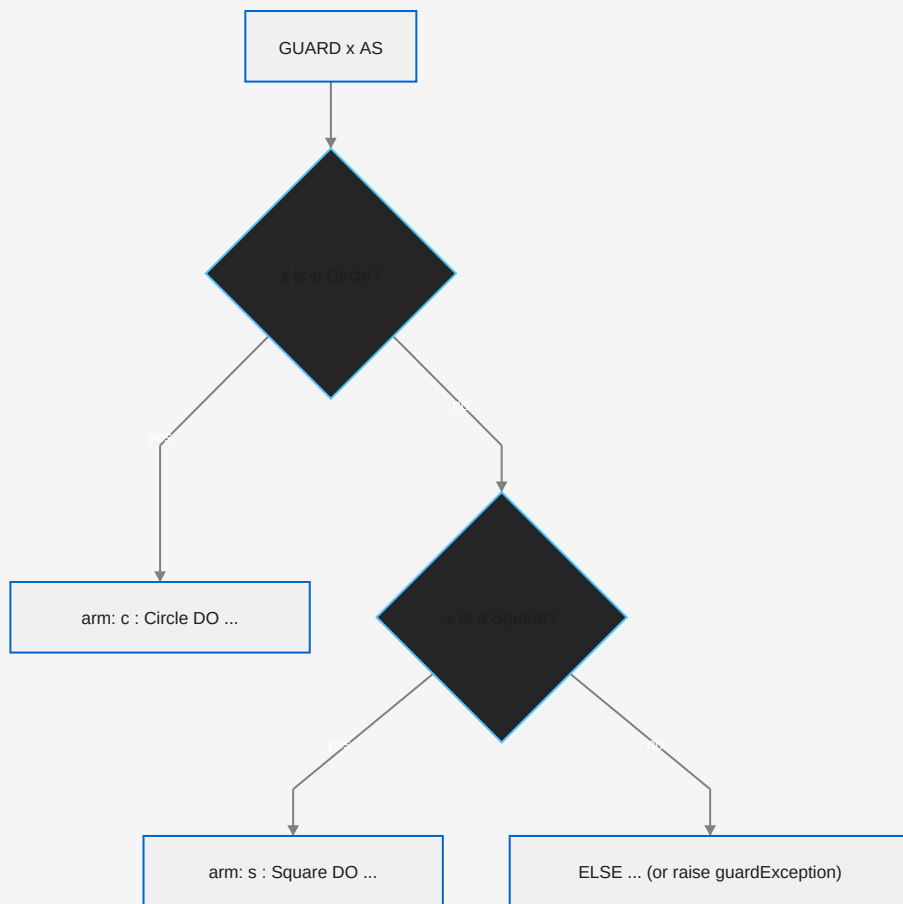
ClassMember = VarDecl ";"
             | [ "ABSTRACT" | "OVERRIDE" ] MethodHeader ";" [ MethodBody ] .
MethodHeader = "PROCEDURE" ident [ FormalParams ] .
```

CLASS, INTERFACE, INHERIT, OVERRIDE, REVEAL, GUARD, and AS are **contextual (soft) keywords** - only ABSTRACT is reserved. See [Objects & classes](#).

Runtime type discrimination

```
GuardStmt  = "GUARD" Expr "AS" GuardArm { "|" GuardArm } [ "ELSE" StatementSeq ] "END" .
GuardArm   = [ ident ":" ] qualident "DO" StatementSeq .

IsMemberCall = "ISMEMBER" "(" objectOrType "," objectOrType ")" .    (* : BOOLEAN *)
```

Back to the manual index. For semantics, follow the chapter links above.